R pour l'archéologue

Exploration, visualisation et analyse de données

Nicolas Frerebeau

2022-02-25

Université Bordeaux Montaigne
Département d'Histoire de l'Art et d'Archéologie

Nicolas Frerebeau

MSHS-Toulouse (UAR 3414)
Maison de la Recherche
Université Fédérale de Toulouse – Jean Jaurès
5 allées Antonio Machado
F-31058 Toulouse cedex 9

Archéosciences Bordeaux (UMR 6034)
Maison de l'Archéologie
Université Bordeaux Montaigne
F-33607 Pessac cedex

Sommaire

Avant propos **v**
Licence . . . v
Contributions . . . vi
Conventions d'écriture . . . vi
Reproductibilité . . . vi

Introduction **1**
Pourquoi travailler avec R ? . . . 1
Ressources complémentaires . . . 2
Pour aller plus loin . . . 4

Installation **7**
Installer R . . . 7
Installer RStudio . . . 7

Premiers pas avec R **11**

1. Prise en main **13**
1.1. Invite de commande . . . 13
1.2. Objets . . . 14
1.3. Fonctions . . . 16
1.4. Packages . . . 18
1.5. Obtenir de l'aide . . . 19

2. Vecteurs **23**
2.1. Vecteurs atomiques (`atomic`) . . . 23
2.2. Listes (`list`) . . . 28
2.3. Matrices (`matrix` et `array`) . . . 31
2.4. Tableaux de données (`data.frame`) . . . 33

3. Extraction et remplacement **35**
3.1. Vecteurs atomiques . . . 35
3.2. Listes . . . 37
3.3. Matrices . . . 40
3.4. Tableaux de données . . . 42

4. Opérateurs binaires **45**
4.1. Opérateurs arithmétiques 45
4.2. Opérateurs logiques 47

5. Structures de contrôle **51**
5.1. Alternatives 51
5.2. Boucles 52

Préparation des données **59**

6. Importer **61**

7. Nettoyer **63**

8. Selectionner **65**

9. Filtrer **67**

10. Transformer **69**

11. Exporter **71**

Éléments de statistiques **73**

12. Statistique descriptive **75**

13. Relations entre variables **77**

14. Visualisation de données **79**

Chronologie **81**

15. Remarques préliminaires **83**

16. Chronologie qualitative **85**

17. Chronologie quantitative **87**

Cartographie **89**

Annexes **91**

A. Fonctions utiles **93**
A.1. Séquences 93

B. Bonnes pratiques **95**
B.1. Travailler par projets 95
B.2. Structurer ses données 96
B.3. Sauvegarder ses données 98
B.4. Limiter les dépendances 100
B.5. Écrire des exemples reproductibles 103

C. Style de programmation **105**
C.1. Dénominations 105
C.2. Syntaxe 108
C.3. Commentaires 114

Bibliographie **117**

Avant propos

Ce livre s'adresse aux personnes qui souhaitent analyser, visualiser et modéliser des données archéologiques à l'aide du langage R. Cet ouvrage vous permet d'acquérir les connaissances et les compétences nécessaires pour aborder un large éventail de problèmes liés aux données archéologiques.

Bien qu'il soit initialement conçu pour les étudiants du master mention *Archéologie, sciences pour l'archéologie* de l'Université Bordeaux Montaigne, ce livre s'adresse à tout étudiant, doctorant, post-doctorant ou archéologue professionnel souhaitant découvrir le langage R et s'y initier pour l'analyse de données archéologiques.

Ce livre s'adresse à des utilisateurs débutants, il ne demande aucun prérequis en programmation. Bien que des notions élémentaires soient rappelées au fil de l'ouvrage, ce livre n'est pas un manuel de statistiques. Le lecteur trouvera des suggestions de lecture complémentaires à la fin de l'introduction[1].

Ce livre est en cours de rédaction.

Licence

Ce livre est mis à disposition selon les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Partage dans les Mêmes Conditions 4.0 International[2].

Le code contenu dans ce livre est simultanément disponible sous la licence MIT[3].

Une entrée BibTeX pour citer ce document :

```
@Book{rarcheo,
  title = {R pour l'archéologue: Exploration, visualisation et analyse de données},
  author = {Nicolas Frerebeau},
```

[1] L'ensemble des références citées dans cet ouvrage est disponible sous la forme d'une bibliographie Zotero

[2] Cela signifie que vous êtes libre de copier, distribuer et communiquer ce livre par tous moyens et sous tous formats, de modifier, transformer et créer à partir du matériel de ce livre. En contrepartie, vous devez créditer la source d'origine en précisant si vous l'avez modifiée, vous devez partager sous la même licence et vous ne pouvez pas en faire un usage commercial.

[3] Cela signifie que vous êtes libre de l'utiliser dans vos propres projets, à condition de citer la source.

```
  year = {2021},
  publisher = {Université Bordeaux Montaigne},
  address = {Pessac, France},
  url = {https://rstats.archaeo.science},
}
```

Contributions

Le code source de ce livre est disponible sur GitHub. N'hésitez pas à y ouvrir un ticket pour signaler toutes les erreurs que vous pourrez constater ou proposer des modifications.

Conventions d'écriture

Par la suite, on écrira `foo()` pour désigner une fonction, `bar` pour désigner les variables et les paramètres des fonctions et `baz/` pour désigner les chemins. Les noms de *packages* sont indiqués en italique.

Les commentaires dans le code sont précédés par le signe `#`. Les blocs de code alternent l'entrée et la sortie. La sortie est commentée (précédée par `#>` pour la distinguer des commentaires standards) de sorte que vous pouvez facilement copier et coller les exemples dans R.

Reproductibilité

Ce livre a été écrit dans RStudio avec le package R *bookdown* (Xie, 2016). Le site web est hébergé par GitHub Pages et automatiquement mis à jour après chaque commit par GitHub Actions (cette version a été compilée le 25 Feb 2022).

Les données utilisées dans les exemples sont issues des packages *datasets* (R Core Team, 2021) et *folio* (Frerebeau, 2021). Les exemples reposant sur la génération de nombre aléatoires sont reproductibles en utilisant `set.seed(12345)`.

Cette version du livre a été réalisée avec R version 4.1.2 (2021-11-01), les packages utilisés sont résumés dans le tableau 1.

Tab. 1. : Liste des packages utilisés.

Package	Version	Source	Référence
bookdown	0.24	CRAN (R 4.1.2)	Xie (2021a)
bslib	0.3.1	CRAN (R 4.1.2)	Sievert et Cheng (2021)

Package	Version	Source	Référence
desc	1.4.0	CRAN (R 4.1.2)	Csárdi et al. (2021)
DiagrammeR	1.0.8	CRAN (R 4.1.2)	Iannone (2022)
downlit	0.4.0	CRAN (R 4.1.2)	Wickham (2021)
folio	1.0.0	CRAN (R 4.1.2)	Frerebeau (2021)
knitr	1.37	CRAN (R 4.1.2)	Xie (2021b)
markdown	1.1	CRAN (R 4.1.2)	Allaire et al. (2019)
mime	0.12	CRAN (R 4.1.2)	Xie (2021c)
rmarkdown	2.11	CRAN (R 4.1.2)	Allaire et al. (2021)
sessioninfo	1.2.2	CRAN (R 4.1.2)	Wickham, Chang et al. (2021)
webshot	0.5.2	CRAN (R 4.1.2)	Chang (2019)
xml2	1.3.3	CRAN (R 4.1.2)	Wickham, Hester et al. (2021)

Introduction

Pourquoi travailler avec R ?

R est libre et gratuit Le développement de R a débuté en 1991 à l'initiative de Ross Ihaka et Robert Gentleman au sein du département de statistiques de l'université d'Auckland (Nouvelle-Zélande). En 1995 la première version officielle est publiée et R devient un logiciel libre, distribué sous licence GNU. Le code source de l'ensemble du système R est ainsi accessible à tous, en garantissant quatre libertés essentielles :

- La liberté de faire fonctionner le programme comme vous voulez, pour n'importe quel usage (liberté 0) ;
- La liberté d'étudier le fonctionnement du programme, et de le modifier pour qu'il effectue vos tâches informatiques comme vous le souhaitez (liberté 1) ; l'accès au code source est une condition nécessaire ;
- La liberté de redistribuer des copies, donc d'aider les autres (liberté 2) ;
- La liberté de distribuer aux autres des copies de vos versions modifiées (liberté 3) ; en faisant cela, vous donnez à toute la communauté une possibilité de profiter de vos changements ; l'accès au code source est une condition nécessaire.

R est modulable Les libertés offertes par la licence GNU et la structure modulaire de R permettent à toute personne de développer des extensions (appelées *packages*) et de les mettre à disposition de la communauté des utilisateurs. De nombreux packages fournissent des interfaces entre le langage R et d'autres langages ou logiciels libres, ce qui en fait un outil polyvalent, une passerelle permettant d'utiliser de nombreuses méthodes et algorithmes spécialisés. Ces packages peuvent être publiés et installés depuis le *Comprehensive R Archive Network*[4] (CRAN) qui distribue également le code source de R et sa documentation. Fin 2021, plus de 22500 packages étaient disponibles sur le CRAN (fig. 1).

R facilite la reproductibilité R est un langage de programmation qui fonctionne à l'aide de scripts. Ces scripts permettent de définir une suite d'instructions à exécuter et contiennent ainsi l'ensemble des étapes de l'analyse. Il est possible de modifier ou de rééxecuter une analyse pour mettre à jour des résultats ou de réutiliser un même script avec des données différentes. Ces scripts peuvent être archivés,

[4] Il existe d'autres archives, parfois spécialisées comme Bioconductor pour la biologie.

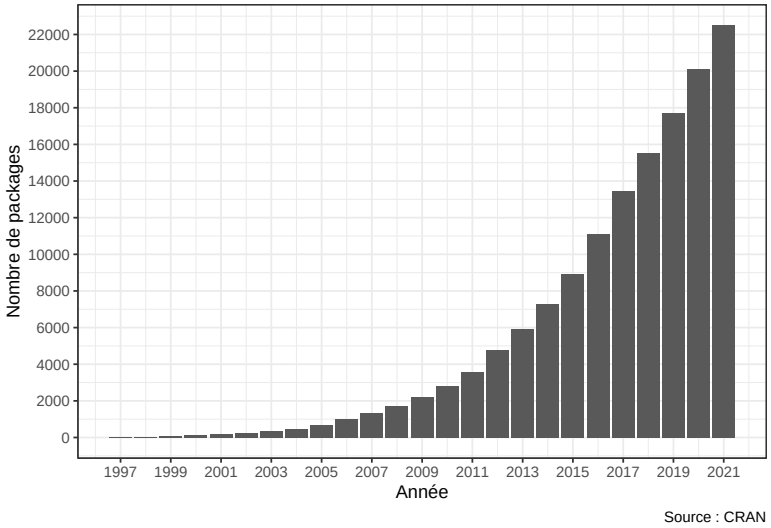

FIG. 1. : Évolution du nombre de packages disponibles sur le CRAN par année. Cette croissance exponentielle n'est pas sans poser de question quant au futur du CRAN (THIEME, 2018).

partagés, soumis à l'examen des pairs et publiés au bénéfice d'une science ouverte, transparente et reproductible.

R s'est ainsi progressivement imposé comme un outil de référence pour l'analyse de données dans de nombreuses disciplines, notamment en archéologie (fig. 2). Si l'apprentissage du langage peut paraître difficile pour qui n'a jamais programmé, R bénéficie d'une communauté très active d'utilisateurs, de contributeurs et de développeurs. La documentation est abondante et il est toujours possible de trouver de l'aide en ligne.

Enfin, l'usage de R, comme de tout autre langage des programmation, conduit très souvent à généraliser un cas particulier. C'est ainsi un très bon moyen pour améliorer sa réflexion et, éventuellement, revoir la façon dont les problèmes sont posés.

Ressources complémentaires

Éléments de statistiques

Utiliser R

En complément de la documentation officielle pour démarrer avec R et comprendre la structure des données, il existe de très nombreux ouvrages dont la plupart sont consultables en ligne. *R in a Nutshell* de ADLER (2010), *Hands-On Programming with R* de GROLEMUND (2015) et *R cookbook* de LONG (2019) couvrent les usages élémentaires

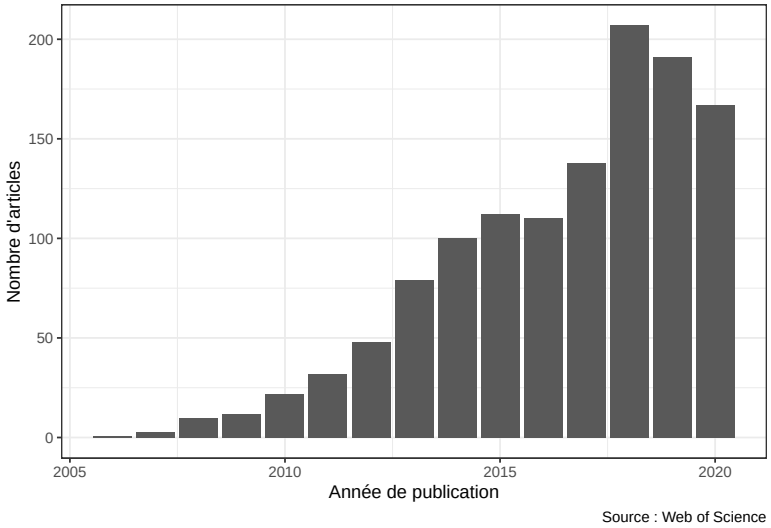

FIG. 2. : Nombre d'articles d'archéologie citant R publié chaque année (répliqué d'après SCHMIDT & MARWICK, 2020).

du langage (plus qu'ils ne sont traités ici) et sont d'une aide précieuse pour un usage courant.

L'ouvrage de référence de MURRELL (2019), *R graphics*, offre une vue d'ensemble des fonctionnalités graphiques de R et de leur usage avancé. *R Graphics Cookbook* (CHANG, 2012) constitue également une référence indispensable et fournit des recettes clé en main pour la création de graphiques. Quiconque s'attachant à la visualisation de données consultera avec intérêt *Fundamentals of Data Visualization* de WILKE (2019).

L'utilisateur confirmé trouvera dans les ouvrages *Advanced R* (WICKHAM, 2019) et *Efficient R programming* (GILLESPIE, 2016) de quoi approfondir sa compréhension du langage et développer sa pratique. *R packages* de WICKHAM (2015) est l'ouvrage idéal pour tout utilisateur souhaitant écrire son premier package et se former aux outils de développement (*devtools*, *roxygen2*, *testthat*...)[5].

Quelques ouvrages thématiques susceptibles d'intéresser l'archéologue[6] :

- *Comprendre et réaliser les tests statistiques à l'aide de R* (MILLOT, 2014), un ouvrage détaillé expliquant comment choisir un test statistique et présentant toutes les subtilités.

[5]Voir également la documentation officielle pour le développement d'extensions de R, la description du processus de création des modules complémentaires, l'écriture de la documentation, les interfaces et l'API : *Writing R Extensions*.

[6]On trouvera sur le site du package bookdown et sur le site du Big Book of R de nombreux ouvrages supplémentaires, librement consultables en ligne. Le site du projet R propose également une liste d'ouvrages dédiés à R et ses applications.

- *Geocomputation with R* (Lovelace, 2020), un livre sur l'analyse, la visualisation et la modélisation des données géographiques.
- *Text Mining with R* (Silge, 2017), un livre sur l'analyse textuelle avec R.

Enfin, on trouvera sur R-bloggers (un agrégateur de blogs) l'actualité du langage et de nombreux tutoriels.

Conseil

Le site collaboratif Rzine a pour objectif de favoriser la diffusion et le partage de connaissances sur la pratique de R en SHS :

> Rzine.fr s'efforce de dessiner les contours de la communauté R en SHS. Il donne aux débutant · e · s un accès simple à l'information et orienté vers une utilisation autonome. Il offre aux utilisateur · rice · s de niveau intermédiaire la possibilité d'étendre leurs pratiques et de s'ouvrir à d'autres méthodes parfois issues d'autres disciplines. Enfin, il s'agit d'un espace de diffusion pour les utilisateur · rice · s avancé · e · s, auteur · e · s de développements ou de documentation. — Rzine

Rzine recense de nombreuses ressources, fiches et manuels, *open source* et rédigés en français. Le site du projet permet également suivre l'actualité du langage et des évènements liés à la pratique de R en SHS.

Pour aller plus loin

Si les enjeux de reproductibilité et de science ouverte vous intéressent, il vous faudra sans doute compléter votre maîtrise de R par des outils supplémentaires, notamment les logiciels de contrôle de version (Bryan, 2021).

Remarque

Le tidyverse est un ensemble de packages partageant "une philosophie de conception, une grammaire et des structures de données" dont les principes sont détaillés par Wickham et al. (2019). Tout comme le tidyverse revendique un "parti pris" (*opinionated*), ce livre revendique un choix : celui de ne pas utiliser les packages du tidyverse, malgré leur popularité au sein de la communauté des utilisateurs de R.

Ce choix repose sur un constat simple : l'usage de ces packages tend à considérablement complexifier l'apprentissage de R par les débutants et à les enfermer dans un écosystème particulier. Au contraire, l'utilisateur qui maîtrise les bases de R peut faire face à une multitudes de situations (il s'agit ici d'apprendre les bases d'un langage, plutôt que celles d'un dialecte particulier[a]).

Ceci étant dit, le lecteur qui le souhaite trouvera dans *Introduction à R et au tidyverse* (Barnier, 2021), *R for Data Science* (Wickham, 2016) et *STAT 545* (Bryan, 2020) une introduction à l'analyse de données avec les packages du tidyverse.

[a] Voir Matloff (2019) pour une discussion détaillée.

Installation

Installer R

R est fonctionne sur les principaux systèmes d'exploitation. Vous pouvez télécharger les fichiers d'installation depuis le site du CRAN. Sous Windows et Mac OS, lancez l'exécutable après l'avoir téléchargé et suivez les étapes d'installation en laissant les options par défaut.

R est disponible dans les dépôts d'Ubuntu. Cependant, les paquets inclus dans les dépôts Ubuntu par défaut sont souvent un peu anciens. Il peut être préférable d'installer R à partir du dépôt du CRAN pour bénéficier des dernières versions.

Pour installer R sur Ubuntu :

```
sudo apt update
sudo apt -y install r-base r-base-dev
```

Important

Des changements importants peuvent subvenir entre les versions de R, entraînant parfois des problèmes de compatibilité. Il est fortement recommandé d'utiliser la dernière version mineure de R et de maintenir son installation à jour.

Installer RStudio

RStudio est un environnement de développement intégré (*interactive development environment*, IDE)[7]. RStudio *n'est pas* R, pas plus qu'il n'offre une interface graphique à R. Il s'agit d'un logiciel utilitaire qui permet d'éditer et d'exécuter votre code, ainsi que de réaliser des actions connexes telles que l'installation de packages ou l'import de données. RStudio dispose notamment d'un système de gestion de projets qui permet d'organiser son travail.

[7] RStudio n'est pas le seul IDE, des utilisateurs avancés peuvent lui préférer *Emacs Speaks Statistics* (ESS), un plugin pour l'éditeur Emacs.

RStudio Desktop est disponible sous licence *open-source* pour les principaux systèmes d'exploitation et peut être téléchargée depuis le site de la société éditrice.

Pour installer RStudio sur Ubuntu, ouvrez un terminal dans le dossier de téléchargement et lancez la commande suivante (le nom du fichier peut différer en fonction de la version) :

```
sudo dpkg -i rstudio-2021.09.2-382-amd64.deb
```

Important

Après l'installation de RStudio, la première étape consiste à en changer les réglages par défaut, pour éviter qu'il ne conserve des éléments entre deux sessions de travail. Ouvrez les options globales (*Tools* > *Global Options…*) et modifiez les réglages suivants (fig. 3) :

- Décochez *Restore .RData into workspace at startup*.
- Réglez *Save workspace to .RData on exit* sur *never*.

Ces deux réglages vous épargneront de mauvaises surprises (et vous éviteront de faire débuter tous vos scripts par `rm(list = ls())`).

L'interface de RStudio se présente sous la forme d'un ensemble d'onglets répartis en quadrants (fig. 4). Par défaut, de haut en bas et de gauche à droite :

- Le premier quadrant contient les documents en cours d'édition (scripts…), permet d'explorer les données et offre un accès rapide à certaines commandes en fonction du contexte (exécuter un script, compiler un document…).
- Le second quadrant permet d'accéder aux objets présent dans l'environnement et de voir l'historique des commandes.
- Le troisième quadrant contient la console.
- Le quatrième quadrant permet de naviguer dans les fichiers, de visualiser les graphiques (générés depuis la console ou à l'aide d'un script), de gérer les packages et d'afficher l'aide.

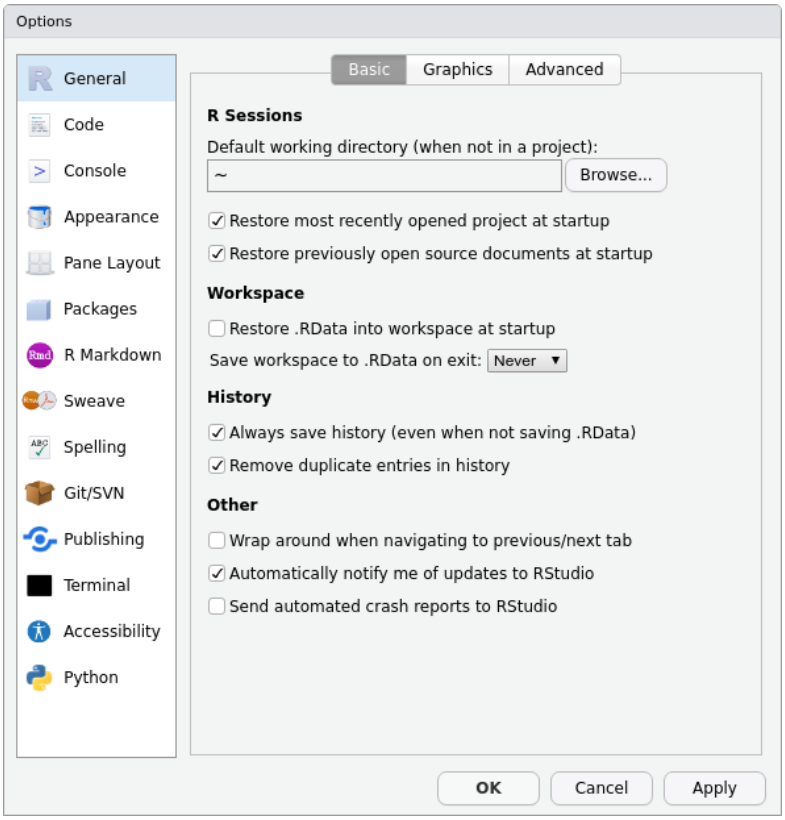

FIG. 3. : Configuration globale de RStudio Desktop.

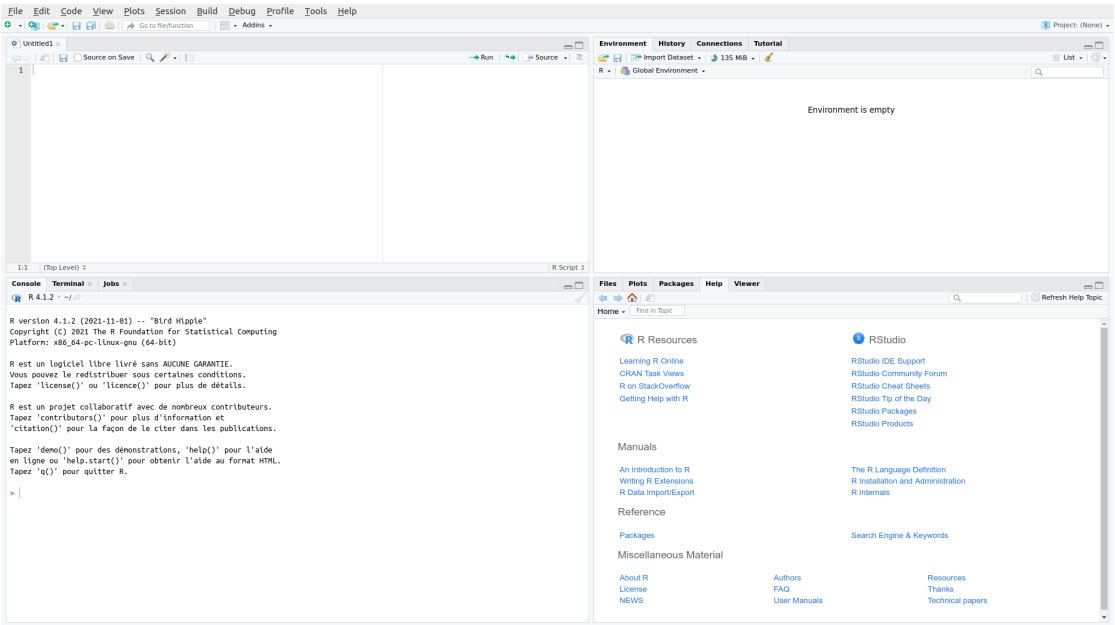

FIG. 4. : L'interface de RStudio Desktop.

Première partie

Premiers pas avec R

1. Prise en main

1.1. Invite de commande

R se présente sous la forme d'une interface en ligne de commande, permettant de donner des instructions à l'ordinateur. Dans RStudio, cette interface est accessible dans l'onglet *Console*.

Lorsque R est en cours d'exécution, l'interface signifie qu'elle est prête à recevoir des instructions par une invite de commande (*prompt*) indiquée par un chevron `>`.

Il est alors possible d'entrer une **expression**, qui va être **évaluée** par R avant d'en retourner le résultat :

```
1 + 1
#> [1] 2
```

Sans surprise, le résultat de l'addition est 2. R donne une information supplémentaire, le `[1]` devant `2` précise qu'il s'agit du premier (dans le cas présent, du seul) élément retourné. Certaines commandes peuvent retourner plusieurs valeurs. Par exemple, l'opérateur `:` (deux-points) permet de construire des séquences de nombres entiers :

```
0:100
#>  [1]   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15
#> [17]  16  17  18  19  20  21  22  23  24  25  26  27  28  29  30  31
#> [33]  32  33  34  35  36  37  38  39  40  41  42  43  44  45  46  47
#> [49]  48  49  50  51  52  53  54  55  56  57  58  59  60  61  62  63
#> [65]  64  65  66  67  68  69  70  71  72  73  74  75  76  77  78  79
#> [81]  80  81  82  83  84  85  86  87  88  89  90  91  92  93  94  95
#> [97]  96  97  98  99 100
```

La commande `0:100` retourne 101 valeurs qui sont affichées sur plusieurs lignes : le nombre entre crochet indique l'**indice** de la valeur par laquelle débute chacune des lignes (la deuxième ligne commence par la 17e valeur, etc.).

Important

Contrairement à Python, dans R les indices commencent à 1 et non à 0.

Si une commande incomplète est transmise, l'invite de commande signale par un `+` que la suite des instructions peut être saisie sur plusieurs lignes :

```
1 -
1
#> [1] 0
```

Conseil

Il est possible d'interrompre la saisie ou d'annuler une commande avec `ctrl/cmd + c`.

Si une commande erronée est transmise, R s'interrompt et retourne un message d'erreur (plus ou moins explicite) :

```
x
#> Error in eval(expr, envir, enclos): object 'x' not found
```

Conseil

Il est possible de naviguer dans l'historique des commandes précédemment exécutées à l'aide des flèches `haut` et `bas` du clavier.

1.2. Objets

Il est possible d'associer un nom à une valeur et ainsi de créer des **variables** (au sens informatique) qui peuvent ensuite être appelées dans l'invite de commande. Une variables n'est rien d'autre qu'un emplacement de la mémoire de l'ordinateur, réservé pour stocker une valeur.

Cette **affectation** (ou assignation, par anglicisme) est réalisée à l'aide de l'opérateur `<-` :

```
x <- 1
x
#> [1] 1
```

Il est possible de copier une variable :

```
y <- x
y
#> [1] 1
```

Une fois copié, il n'existe aucun lien entre l'objet initial et sa copie. Dans l'exemple précédent, si `x` est modifié après avoir été copié dans `y`, la valeur de `y` ne sera pas modifiée.

Il est possible d'affecter plusieurs variables en même temps :

```
i <- j <- 1
```

Les variables ainsi crées peuvent être réutilisées :

```
x + 1
#> [1] 2
i + j
#> [1] 2
```

La réutilisation d'un nom de variable existant entraîne le remplacement de la valeur précédente (sans avertissement) :

```
i <- 1
i
#> [1] 1

i <- 2
i
#> [1] 2
```

Important

- Un nom de variable peut uniquement être composé de lettres, de chiffres et des caractères `.` ou `_`.
- Un nom de variable doit toujours commencer par une lettre (ou par un point suivi d'une lettre).
- Les noms de variables sont sensibles à la casse (`x` et `X` sont deux variables différentes).
- Il existe des mots réservés qui ne peuvent être utilisés comme nom de variable.

Dans R les variables stockent des **objets**[1]. Ainsi, le code `x <- 1` peut se lire "créer un objet nommé x dont la valeur est 1"[2].

[1] En informatique, un objet est une représentation d'une entité du monde réel et de ses attributs, auxquels sont associés des traitements particuliers.

[2] Il s'agit cependant d'une simplification, voir la section "Binding basics" dans Wickham (2019) pour plus de détails.

Un objet possède un type et une structure de données particulière. Le type d'un objet est lié au type d'information qu'il contient et à la façon dont il est stocké dans la mémoire[3]. La structure de données d'un objet correspond à la manière dont sont organisées les données.

1.3. Fonctions

Il existe des objets particuliers, qui permettent d'agir sur d'autres objets : les **fonctions**. Ces fonctions peuvent accepter une ou plusieurs valeurs (ou objets) en entrée, appelés **arguments**, et retournent un objet au terme de leur exécution. Les arguments permettent de modifier le comportement d'une fonction.

R fournit de très nombreuses fonctions, mais pour réaliser des taches très spécifiques il est possible d'écrire ses propres fonctions ou d'installer des packages supplémentaires.

1.3.1. Utiliser des fonctions

Par exemple, la fonction `round()` permet d'arrondir une valeur numérique au nombre de décimales spécifié. Par défaut, la fonction `round()` réalise un arrondi à zéro chiffre après la virgule :

```
round(3.141593)
#> [1] 3
```

La fonction `args()` permet de connaitre les différents arguments d'une fonction et les valeurs par défaut qui peuvent leur être associées :

```
args(round)
#> function (x, digits = 0)
#> NULL
```

La fonction `round()` accepte ainsi deux arguments, la valeur à arrondir et le nombre de décimales à conserver (0 par défaut).

Remarque

Les arguments qui possèdent une valeur par défaut sont des **arguments facultatifs** : ils peuvent être omis lorsque la fonction est appelée (la valeur par défaut sera utilisée). À l'inverse, les arguments sans valeurs par défaut doivent être spécifiés : ce sont des **arguments obligatoires**.

[3] Votre système d'exploitation alloue la mémoire en fonction du type de donnée et décide de ce qui peut être stocké dans la mémoire réservée lors de la création d'une variable.

Les différents arguments d'une fonction sont séparés par une virgule et sont nommés. Lors de l'appel d'une fonction, si les arguments ne sont pas spécifiés par leur nom, R va associer les valeurs correspondantes dans l'ordre. Dans l'exemple suivant, la valeur 2 est associée au second argument (`digits`) :

```
round(3.141593, 2)
#> [1] 3.14
```

Pour éviter les erreurs et pour faciliter la lecture du code, dans un appel de fonction, spécifiez toujours les arguments par leur nom (à l'exception du premier argument). Les arguments obligatoires doivent être placés en premier, suivis des arguments facultatifs :

```
round(3.141593, digits = 2)
#> [1] 3.14
```

Conseil

Le nom des arguments d'une fonction, leurs rôles et leurs éventuelles valeurs par défaut sont détaillés dans l'aide.

1.3.2. Écrire des fonctions

Comme son nom le laisse deviner, fonction `function()` permet de créer ses propres fonctions.

```
addition <- function(x, y) {
  # On définit les instructions à exécuter
  z <- x + y
  # On précise le résultat à retourner au terme des calculs
  return(z)
}

addition(1, 2)
#> [1] 3
```

Il est possible de définir des arguments facultatifs et ainsi de spécifier le comportement par défaut d'une fonction.

La fonction suivante permet d'élever un nombre à la puissance désirée. En spécifiant la valeur du second argument (qui correspond à l'exposant) dans la définition de la fonction, on peut prédéfinir son comportement : par défaut, notre fonction va élever le nombre désiré au carré :

```r
# Définition de la fonction
# Par défaut, la valeur de y est 2
puissance <- function(x, y = 2) {
  z <- x^y
  return(z)
}

# L'utilisateur ne change pas la valeur par défaut
puissance(2) # 2 au carré
#> [1] 4

# L'utilisateur change explicitement la valeur par défaut
puissance(2, 3) # 2 au cube
#> [1] 8
```

1.4. Packages

Lors d'une première installation de R, un ensemble de packages contenant les fonctions fondamentales est installé :

- Les packages essentiels : `base`, `compiler`, `datasets`, `graphics`, `grDevices`, `grid`, `methods`, `parallel`, `splines`, `stats`, `stats4`, `tcltk`, `tools`, `translations`, `utils`.
- Les packages recommandés : `boot`, `class`, `cluster`, `codetools`, `foreign`, `KernSmooth`, `lattice`, `MASS`, `Matrix`, `mgcv`, `nlme`, `nnet`, `rpart`, `spatial`, `survival`.

L'installation d'un package permet de bénéficier de fonctionnalités supplémentaires, généralement dédiées à une tâche bien spécifique. Pour ne pas se perdre dans la multitude de ressources disponibles, le CRAN propose des répertoires de packages par domaine (*Task Views*) pour faciliter le choix des packages pour une analyse spécifique. Il existe ainsi des répertoires pour les sciences sociales, l'analyse de données environnementales ou encore l'analyse de données spatiales. Pour l'archéologie, il existe une *Task View* non officielle, maintenue par Ben Marwick.

Conseil

Une bonne pratique consiste à toujours chercher à limiter les dépendances.

Lorsqu'un package est disponible sur le CRAN, il peut aisément être installé à l'aide de la fonction `install.packages()`[4].

La commande suivante permet d'installer le package *folio* qui contient les jeux de données utilisés par la suite :

```
install.packages("folio")
```

Conseil

La plupart des packages publiés sur le CRAN font l'objet de mises à jour régulières. Pour maintenir à jour les packages installés sur votre ordinateur, la fonction `old.packages()` permet de lister les packages pour lesquels il existe une nouvelle version et `update.packages()` permet de télécharger et d'installer les nouvelles versions.

Installer un nouveau package est une condition nécessaire, mais pas suffisante, pour pouvoir l'utiliser. Au lancement, R ne charge pas tous les packages installés, mais uniquement les packages de base[5]. Dans l'exemple suivant, si on utilise la fonction `data()` pour charger le jeux de données `intcal20` du package *folio* sans que ce dernier soit chargé. R ne sait pas où chercher l'objet et retourne un avertissement :

```
data(intcal20)
```

Il est donc nécessaire de charger un package à l'aide de la fonction `library()` avant de pouvoir l'utiliser[6] :

```
library("folio")
data(intcal20)
```

1.5. Obtenir de l'aide

Lorsque vous ne savez pas comment utiliser une fonction particulière (difficile de mémoriser les 1172 fonctions des packages de base), votre premier réflexe doit être de consulter la documentation. La documentation couvre l'ensemble des fonctions, jeux de données et autres objets présents dans les différents packages.

[4] `install.packages()` permet également d'installer des packages publiés sur d'autres dépôts que le CRAN, comme Bioconductor.

[5] La liste des packages chargés par défaut au lancement de R peut être obtenue avec `getOption('defaultPackages')`.

[6] Dans le cas présent, une alternative serait de spécifier l'argument `package` de la fonction `data()`.

Pour un objet donné, on peut consulter la documentation associée en utilisant soit la fonction `help()`, soit l'opérateur `?`. Par exemple, pour accéder à l'aide de la fonction `round()` :

```
help(round)

?round
```

Si l'objet recherché appartient à un package qui n'est pas chargé, R ne saura pas où chercher la documentation. Dans l'exemple suivant, la fonction `lda()` appartient au package *MASS* qui n'est pas chargé :

```
help(lda)
#> No documentation for 'lda' in specified packages and libraries:
#> you could try '??lda'
```

Pour accéder à la documentation, il faudra alors charger le package au préalable ou le spécifier dans la fonction `help()` :

```
help(lda, package = "MASS")
```

Si on ignore le nom de la fonction à utiliser, deux approches permettent d'explorer la documentation :

- On peut parcourir la documentation d'un package particulier, dont on sait qu'il doit contenir la fonction recherchée (par exemple, `help(package = "MASS")` permet d'afficher le sommaire de la documentation du package *MASS*).
- On peut rechercher dans l'aide par mot-clé à l'aide de l'opérateur `??` ou de la fonction `help.search()`.

Par exemple, pour chercher toutes les pages de la documentation contenant le terme "correlation" :

```
help.search("correlation")

??correlation
```

Toutes les pages d'aide sont structurées de la même façon. Généralement, on y trouve au moins les sections suivantes :

Description Une courte description de ce que fait la fonction.

Usage Un exemple d'utilisation de la fonction présentant l'ensemble des arguments et leurs éventuelles valeurs par défaut.

Arguments La liste des arguments de la fonction précisant leur nom, leur rôle et la valeur ou le type d'objet qui est attendu.
Details Une explication détaillée du comportement de la fonction.
Value Une description de ce qui est retourné lorsque la fonction est exécutée.
References Les références bibliographiques associées à la fonction.
See Also Une suggestion de fonctions similaires.
Examples Quelques exemples illustrant différents cas d'utilisation de la fonction. Ces exemples peuvent être exécutés directement avec la fonction `example()`.

Pour exécuter les exemples illustrant la documentation de la fonction `mean()` :

```
example(mean)
```

Conseil

Si vous faites face à une difficulté, il fort probable quelqu'un se soit déjà posé la même question que vous et que la réponse se trouve sur Stack Overflow ou sur les listes de diffusions R-help et R-devel (actives depuis 1996). Dans le cas contraire, vous pourrez y exposer votre problème, mais il vous faudra apprendre à écrire des exemples reproductibles.

2. Vecteurs

Le vecteur (`vector`) est la structure de base de R, à partir de laquelle toutes les autres sont construites (fig. 2.1). Un vecteur est simplement un ensemble d'**éléments**.

Un vecteur peut être un vecteur atomique (`atomic`), si tous ses éléments sont de même **type**, ou une liste (`list`), si ses éléments sont de types différents (tab. 2.1). Un vecteur peut également avoir des **attributs** (sorte de métadonnées).

Fig. 2.1. : Les types d'objets de R. D'après Wickham (2019).

Tab. 2.1. : Structures de données des objets de R.

Dimensions	Objet homogène	Objet hétérogène
1	`atomic`	`list`
2	`matrix`	`data.frame`
n	`array`	-

2.1. Vecteurs atomiques (`atomic`)

Un vecteur atomique possède deux propriétés : son type et sa longueur (le nombre d'éléments qu'il contient). Les quatre principaux types de R sont les suivants :

- `logical` : booléen.
- `integer` : nombre entier.
- `double` : nombre réel, avec ou sans partie décimale.
- `character` : chaîne de caractère.

2.1.1. Création

Chacun de ces types possède une syntaxe particulière pour former des vecteur contenant un unique élément (scalaires) :

```
x <- TRUE          # logical
x <- 1L            # integer
x <- 10.5          # double
x <- "archéologie" # character
```

- Un vecteur logique ne peut pendre que deux valeurs `TRUE` (vrai) ou `FALSE` (faux). `TRUE` et `FALSE` sont des mots réservés dans R : ils ne peuvent pas être utilisés comme identifiants pour nommer des variables ou des fonctions.
- Un nombre réel sans partie décimale sera toujours interprété comme étant de type `double` et non de type `integer`.
- Un vecteur de type `integer` s'écrit avec une syntaxe particulière : les nombres doivent être suivis d'un `L` pour les distinguer du type `double` et ne doivent pas avoir de partie décimale.
- Une chaîne de caractère doit toujours être encadrée par des guillemets simples (`'`) ou doubles (`"`). Si les guillemets sont manquantes, R l'interprétera comme un nom d'objet (et cherchera l'objet correspondant).

La fonction `c()` (*combine*) permet de construire des vecteurs longs :

```
vec_log <- c(TRUE, FALSE, TRUE) # logical
vec_int <- c(1L, 2L, 3L)        # integer
vec_num <- c(1.2, 2.1, 3.2)     # double
vec_cha <- c("A", "B", "C")     # character
```

2.1.2. Description

La fonction `typeof()` permet de connaitre le type d'un vecteur et la fonction `length()` sa longueur :

```
typeof(vec_cha)
#> [1] "character"

length(vec_cha)
#> [1] 3
```

La famille de fonctions `is.*()` permet de tester si un vecteur est d'un type particulier :

```
is.logical(vec_log)
#> [1] TRUE

is.integer(vec_int)
#> [1] TRUE

is.double(vec_num)
#> [1] TRUE

is.character(vec_cha)
#> [1] TRUE
```

2.1.3. Conversion

Tous les éléments d'un vecteur atomique sont du même type. Lors de la combinaison d'éléments de types différents, R va réaliser une conversion implicite pour tenter de représenter tous les éléments du vecteur de manière raisonnable. Le type du vecteur final est alors déterminé par la hiérarchie `logical` < `integer` < `double` < `character`.

Un vecteur contenant au moins une chaîne de caractères sera donc de type `character` :

```
x <- c(1, "B", 3)
typeof(x)
#> [1] "character"

x <- c(TRUE, "B", TRUE)
typeof(x)
#> [1] "character"
```

Un vecteur contenant au moins un nombre réel sera de type `double` (même s'il ne possède pas de partie décimale) :

```
x <- c(1L, 2.1, 3L)
typeof(x)
#> [1] "double"

x <- c(TRUE, 2.1, TRUE)
typeof(x)
#> [1] "double"
```

La création d'un `integer` avec une partie décimale sera automatiquement converti en type `double` avec un avertissement :

```
x <- 10.5L
typeof(x)
#> [1] "double"
```

Il est possible de convertir explicitement un vecteur d'un type à l'autre à l'aide de la famille de fonctions `as.*()` :

```
as.integer(c(1.2, 2.1, 3.2))
#> [1] 1 2 3

as.numeric(c(TRUE, FALSE, TRUE))
#> [1] 1 0 1

as.character(c(1.2, 2.1, 3.2))
#> [1] "1.2" "2.1" "3.2"
```

Les valeurs d'un vecteur numérique peuvent être converties en valeurs logiques : les valeurs égales à 0 deviennent `FALSE`, tout autre valeur non nulle devient `TRUE` :

```
as.logical(c(1, 0, 3))
#> [1]  TRUE FALSE  TRUE
```

En cas de conversion impossible, des valeurs manquantes sont générées et un avertissement est retourné :

```
as.numeric(c("A", "B", "C"))
#> Warning: NAs introduits lors de la conversion automatique
#> [1] NA NA NA
```

2.1.4. Attributs

Les attributs d'un vecteur peuvent être vus comme des métadonnées : des données qui fournissent des informations sur les données du vecteur.

Les attributs peuvent être spécifiés individuellement à l'aide de la fonction `attr()`, sous la forme de paires associant un nom et une valeur. Dans l'exemple suivant, deux attributs nommés `a` et `b` sont associés au vecteur `x` avec pour valeurs `"xyz"` et `1 2 3`, respectivement :

```
x <- 1:5

attr(x, "a") <- "xyz"
attr(x, "a")
#> [1] "xyz"

attr(x, "b") <- 1:3
attr(x, "b")
#> [1] 1 2 3
```

Remarque

La syntaxe particulière `attr(x, "a") <- "xyz"` indique qu'il s'agit d'une fonction de remplacement. Ces fonctions particulières modifient en place l'objet correspondant au premier argument (`x`).

L'utilisation des attributs requiert un peu de prudence, car à quelques exceptions près les attributs ne sont pas conservés lors d'opérations sur les vecteurs (comme lors de l'extraction de sous-ensembles). Il existe cependant deux attributs particuliers qui sont généralement conservés : les noms (`names`) et la dimension (`dim`, qui transforme un vecteur en matrice).

Il est possible d'associer des noms aux différents éléments d'un vecteur, soit au moment de sa création, soit *a posteriori* avec la fonction `names()` (qui permet également d'extraire les noms) :

```
# Un nom est associé à chaque élément
x <- c(a = 1, b = 2, c = 3, d = 4, e = 5)
x
#> a b c d e
#> 1 2 3 4 5

# Les noms sont spécifiés à l'aide d'un vecteur
names(x) <- c("f", "g", "h", "i", "j")
x
#> f g h i j
#> 1 2 3 4 5

# Extraire les noms des différents éléments
names(x)
#> [1] "f" "g" "h" "i" "j"

# Supprimer les noms
```

```
names(x) <- NULL
names(x)
#> NULL

x
#> [1] 1 2 3 4 5
```

Remarque

L'utilisation des attributs permet de créer des vecteurs particuliers, appelés `factor`, qui représentent des variables qualitatives (nominales ou ordinales).

2.2. Listes (`list`)

Une liste est un vecteur particulier, dont les éléments peuvent être de n'importe quel type. Une liste peut être créée avec avec la fonction `list()` :

```
# Création d'une liste contenant 4 éléments
x <- list(
  TRUE,
  1L,
  c(1.2, 2.1),
  c("X", "Y", "Z")
)
x
#> [[1]]
#> [1] TRUE
#>
#> [[2]]
#> [1] 1
#>
#> [[3]]
#> [1] 1.2 2.1
#>
#> [[4]]
#> [1] "X" "Y" "Z"

# Une liste est un objet de type 'list'
typeof(x)
#> [1] "list"
```

```
# Longueur de la liste (nombre d'éléments)
length(x)
#> [1] 4

# Longueur des différents éléments de la liste
lengths(x)
#> [1] 1 1 2 3

# Les éléments de la liste peuvent être nommés
names(x) <- c("a", "b", "c", "d")
x
#> $a
#> [1] TRUE
#>
#> $b
#> [1] 1
#>
#> $c
#> [1] 1.2 2.1
#>
#> $d
#> [1] "X" "Y" "Z"
```

Les listes permettent ainsi de créer et de manipuler des objets complexes, qui peuvent contenir d'autres listes (on parle alors de vecteurs récursifs) :

```
x <- list(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  c(1, 2)
)
x
#> [[1]]
#> [[1]][[1]]
#> [1]  TRUE FALSE
#>
#> [[1]][[2]]
#> [1] "X" "Y" "Z"
#>
#>
```

```
#> [[2]]
#> [1] 1 2
```

La fonction `c()` permet de combiner plusieurs listes en une seule :

```
x <- c(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  list(1, 2)
)
x
#> [[1]]
#> [1]  TRUE FALSE
#>
#> [[2]]
#> [1] "X" "Y" "Z"
#>
#> [[3]]
#> [1] 1
#>
#> [[4]]
#> [1] 2
```

Un vecteur atomique est converti en liste avant d'être combiné :

```
x <- c(
  list(
    c(TRUE, FALSE),
    c("X", "Y", "Z")
  ),
  c(1, 2)
)
x
#> [[1]]
#> [1]  TRUE FALSE
#>
#> [[2]]
#> [1] "X" "Y" "Z"
#>
#> [[3]]
```

```
#> [1] 1
#>
#> [[4]]
#> [1] 2
```

2.3. Matrices (matrix et array)

Un vecteur peut être transformé en un tableau à deux (`matrix`) ou plusieurs (`array`) dimensions, en spécifiant l'attribut `dim`. Une matrice, au sens mathématique, est un tableau rectangulaire à deux dimensions : une matrice possède m lignes et n colonnes et contient $m \times n$ éléments du même type.

```
# Soit un vecteur contenant 12 éléments
x <- 1:12
x
#>  [1]  1  2  3  4  5  6  7  8  9 10 11 12

# Le vecteur est transformé en une matrice de 2 lignes et 6 colonnes
dim(x) <- c(2, 6)
dim(x)
#> [1] 2 6
x
#>      [,1] [,2] [,3] [,4] [,5] [,6]
#> [1,]    1    3    5    7    9   11
#> [2,]    2    4    6    8   10   12
```

Les caractéristiques d'une matrice peuvent être retrouvées :

```
typeof(x) # Type des éléments
#> [1] "integer"
length(x) # Nombre d'éléments (valeurs)
#> [1] 12
nrow(x)   # Nombre de lignes
#> [1] 2
ncol(x)   # Nombre de colonnes
#> [1] 6
```

La fonction `matrix()` permet de créer des matrices en spécifiant le nombre de lignes et de colonnes :

```
# Par défaut, une matrice est remplie colonne par colonne
x <- matrix(1:12, nrow = 3, ncol = 4)
x
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12

# Ce comportement peut être modifié
y <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
y
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    2    3    4
#> [2,]    5    6    7    8
#> [3,]    9   10   11   12
```

Plusieurs matrices peuvent être combinées en une seule, par lignes (`rbind()`) ou par colonnes (`cbind()`) :

```
# Combine les lignes de deux matrices
rbind(x, y)
#>      [,1] [,2] [,3] [,4]
#> [1,]    1    4    7   10
#> [2,]    2    5    8   11
#> [3,]    3    6    9   12
#> [4,]    1    2    3    4
#> [5,]    5    6    7    8
#> [6,]    9   10   11   12

# Combine les colonnes de deux matrices
cbind(x, y)
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
#> [1,]    1    4    7   10    1    2    3    4
#> [2,]    2    5    8   11    5    6    7    8
#> [3,]    3    6    9   12    9   10   11   12
```

Enfin, il est possible de spécifier/extraire les noms de lignes et de colonnes avec les fonctions `rownames()` et `colnames()` ou avec la fonction `dimnames()` :

```
rownames(x) <- c("X", "Y", "Z")
colnames(x) <- c("A", "B", "C", "D")
dimnames(x)
```

```
#> [[1]]
#> [1] "X" "Y" "Z"
#>
#> [[2]]
#> [1] "A" "B" "C" "D"

x
#>   A B C  D
#> X 1 4 7 10
#> Y 2 5 8 11
#> Z 3 6 9 12

dimnames(y) <- list(c("X", "Y", "Z"), c("A", "B", "C", "D"))
rownames(y)
#> [1] "X" "Y" "Z"
colnames(y)
#> [1] "A" "B" "C" "D"

y
#>   A  B  C  D
#> X 1  2  3  4
#> Y 5  6  7  8
#> Z 9 10 11 12
```

2.4. Tableaux de données (data.frame)

Le data.frame est probablement le concept le plus important de R : il permet de représenter un tableau de données rectangulaire dont les colonnes peuvent contenir des informations de différentes natures. Un data.frame est un cas particulier de list (fig. 2.1) dont tous les éléments sont nommés et ont la même longueur : chaque élément correspond à une colonne du tableau[1]. Chaque colonne d'un data.frame peut ainsi être d'un type différent (contrairement aux matrices dont tous les éléments sont du même type) :

```
x <- data.frame(
  a = 1:3,
  b = c("x", "y", "z")
)
x
#>   a b
```

[1] Ainsi, la fonction names() est équivalente à rownames() et la fonction length() retourne le nombre de colonne du tableau (équivalent à ncol()).

```
#> 1 1 x
#> 2 2 y
#> 3 3 z
```

Un `data.frame` est une liste :

```
typeof(x) # Type list
#> [1] "list"
length(x) # Nombre d'éléments (colonnes)
#> [1] 2
```

Un `data.frame` est un tableau à deux dimensions :

```
nrow(x) # Nombre de lignes
#> [1] 3
ncol(x) # Nombre de colonnes
#> [1] 2
```

Lignes et colonnes d'un `data.frame` sont nommées :

```
rownames(x) # Noms de lignes
#> [1] "1" "2" "3"
colnames(x) # Noms de colonnes
#> [1] "a" "b"
```

Remarque

Sauf instruction spécifique lors de la création d'un `data.frame`, les noms de lignes sont générés automatiquement à partir d'une séquence de nombres entiers. Malgré les apparences, il s'agit bien de noms de lignes (de type `character`) et ils ne doivent pas être confondu avec les indices des lignes.

Comme pour les matrices, il est possible de combiner les lignes ou les colonnes de plusieurs `data.frame` en un seul tableau de données avec `rbind()` et `cbind()`. Dans le cas de la combinaison de colonnes, il pourra être préférable de réaliser des jointures.

3. Extraction et remplacement

R possède un mécanisme très puissant pour naviguer parmi les valeurs d'un objet, afin d'en extraire ou d'en remplacer tout ou partie. Ce mécanisme obéit cependant à des règles parfois complexes et peut aboutir à des résultats inattendus si elles ne sont pas bien comprises.

La façon la plus simple de sélectionner ou d'extraire des valeurs consiste à utiliser les opérateurs `[` ou `[[` : `[` permet d'extraire un sous-ensemble, tandis que `[[` permet d'extraire un unique élément. Cette distinction correspond aux relations d'appartenance et d'inclusion de la théorie des ensembles en mathématiques[1] (cette distinction peut paraître un peu obscure au premier abord, mais prend tout son sens lors de la manipulation de listes).

Ces opérateurs peuvent être utilisés conjointement avec l'affectation (`<-`) pour remplacer des valeurs. On écrira ainsi : `x[i] <- y` où `i` correspond à l'indice (position) des valeurs à remplacer dans le vecteur `x` et `y` correspond au vecteur de nouvelles valeurs.

3.1. Vecteurs atomiques

Il existe différentes façons d'extraire un sous-ensemble d'éléments au sein d'un vecteur à l'aide de l'opérateur `[` :

```
x <- c(1.3, 2.4, 3.5, 4.6, 5.7)
```

- Il est possible de spécifier les indices (positions) des éléments à conserver ou à enlever avec des nombres entiers :

```
# Sélection du premier, troisième et cinquième élément
x[c(1, 3, 5)]
#> [1] 1.3 3.5 5.7

# Suppression du second et du quatrième élément
x[-c(2, 4)]
#> [1] 1.3 3.5 5.7
```

[1] Par exemple, si M est un ensemble (collection) tel que $M = \{1, 2, 3\}$, alors 1, 2 et 3 sont des éléments de M, tandis que $\{1, 2\}$ ou $\{3\}$ sont des sous-ensembles de M.

```
# Zéro est un cas particulier
# (retourne un vecteur de longueur nulle)
x[0]
#> numeric(0)
```

- Il est possible de spécifier si oui (`TRUE`) ou non (`FALSE`) on souhaite conserver des éléments :

```
x[c(TRUE, FALSE, TRUE, FALSE, TRUE)]
#> [1] 1.3 3.5 5.7
```

- Si rien n'est spécifié, le vecteur d'origine est retourné :

```
x[]
#> [1] 1.3 2.4 3.5 4.6 5.7
```

- Si les éléments sont nommés, on peut les sélectionner par leur nom :

```
names(x) <- c("a", "b", "c", "d", "e")
x[c("a", "c", "e")]
#>   a   c   e
#> 1.3 3.5 5.7
```

L'utilisation de `[[` permet d'extraire un seul et unique élément au lieu d'un sous-ensemble. Dans le cas des vecteurs atomiques, la différence entre `[` et `[[` n'est pas évidente[2] et est sans doute mieux illustrée avec un exemple :

```
x[1] # L'attribut names est conservé (sous-ensemble)
#>   a
#> 1.3

x[[1]] # L'attribut names n'est pas conservé (élément)
#> [1] 1.3
```

Les éléments sélectionnés peuvent être remplacés en affectant de nouvelles valeurs :

```
# Remplacement du premier et du troisième élément
x[c(1, 3)] <- c(100, 300)
x
#>     a     b     c     d     e
#> 100.0   2.4 300.0   4.6   5.7
```

Lors de l'affectation de nouvelles valeurs dans un vecteur existant, il faut veiller à plusieurs aspects :

[2] Hormis qu'une tentative de sélection de plusieurs éléments avec `[[` produira une erreur.

- La duplication d'indices :

```
x <- 1:5
x[c(1, 1, 1)] # Duplique les valeurs
#> [1] 1 1 1
```

- L'ordre des indices :

```
x <- 1:5
x[c(5, 3, 1)] # Change l'ordre des valeurs
#> [1] 5 3 1
```

- Les mécanismes de conversions implicites :

```
x <- 1:5
typeof(x)
#> [1] "integer"

x[c(1, 3)] <- c(100.1, 300.3)
typeof(x)
#> [1] "double"
```

- La taille du vecteur de remplacement. Si ce dernier est trop petit, il sera **recyclé** : ses valeurs seront réutilisées autant que nécessaire :

```
x <- 1:5
x[c(1, 2, 3, 4)] <- c(100, 200)
x
#> [1] 100 200 100 200   5
```

3.2. Listes

L'extraction de sous-ensembles dans une liste s'opère de la même façon que pour un vecteur atomique :

```
x <- list(
  a = c(TRUE, FALSE),
  b = c(1, 2, 3, 4, 5),
  c = c("X", "Y", "Z")
)
x
#> $a
#> [1]  TRUE FALSE
#>
#> $b
```

```
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"

# Sélection du premier et du troisième élément
x[c(1, 3)] # Par position
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"

x[c("a", "c")] # Par nom
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"
```

L'opérateur `[` retourne toujours une liste, y compris lorsqu'un unique élément est sélectionné. Pour extraire un unique élément *hors* de la liste, il est possible d'utiliser l'opérateur `[[` :

```
y <- x[1]
y
#> $a
#> [1]  TRUE FALSE
typeof(y)
#> [1] "list"

z <- x[[1]]
z
#> [1]  TRUE FALSE
typeof(z)
#> [1] "logical"
```

Si les éléments de la liste sont nommés, on peut extraire un unique élément en le sélectionnant par son nom avec `[[` ou avec l'opérateur `$` (dollar) :

```
x[["a"]]
#> [1]  TRUE FALSE
```

```
x$a
#> [1]  TRUE FALSE
```

Cette distinction entre `[` et `[[` est particulièrement importante lorsqu'on souhaite remplacer des valeurs dans une liste (puisque `[` retourne toujours une liste, la valeur de remplacement doit également être une liste) :

```
x[1] <- list(1:10)
x
#> $a
#>  [1]  1  2  3  4  5  6  7  8  9 10
#>
#> $b
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"

x[[1]] <- c(TRUE, FALSE)
x
#> $a
#> [1]  TRUE FALSE
#>
#> $b
#> [1] 1 2 3 4 5
#>
#> $c
#> [1] "X" "Y" "Z"
```

L'opérateur `[[` permet également de supprimer un élément dans une liste en lui affectant la valeur `NULL` :

```
x[[2]] <- NULL
x
#> $a
#> [1]  TRUE FALSE
#>
#> $c
#> [1] "X" "Y" "Z"
```

3.3. Matrices

La sélection d'un sous-ensemble dans un tableau à deux dimensions (`matrix` ou `array`) est une généralisation de la sélection dans un vecteur. Il suffit de spécifier les indices des lignes et des colonnes, séparés par une virgule :

```
x <- matrix(1:12, nrow = 3, ncol = 4)
rownames(x) <- c("x", "y", "z")
colnames(x) <- c("A", "B", "C", "D")
x
#>   A B C  D
#> x 1 4 7 10
#> y 2 5 8 11
#> z 3 6 9 12

# Sélection des lignes 2 à 3 et des colonnes 1 à 3
x[2:3, 1:3]
#>   A B C
#> y 2 5 8
#> z 3 6 9

x[-1, -4]
#>   A B C
#> y 2 5 8
#> z 3 6 9

x[c(FALSE, TRUE, TRUE), c("A", "B", "C")]
#>   A B C
#> y 2 5 8
#> z 3 6 9
```

Si rien n'est spécifié, toutes les lignes ou toutes les colonnes sont sélectionnées :

```
# Sélection de toutes les lignes et des colonnes 1 à 3
x[, 1:3]
#>   A B C
#> x 1 4 7
#> y 2 5 8
#> z 3 6 9

# Sélection de toutes les colonnes et des lignes 2 à 3
x[2:3, ]
#>   A B C  D
```

```
#> y 2 5 8 11
#> z 3 6 9 12
```

Par défaut, `[` **simplifie** les résultats à la plus petite dimension possible. La sélection d'une unique valeur, d'une unique ligne ou d'une unique colonne, ne retourne donc pas une matrice mais un vecteur. La structure de l'objet d'origine peut cependant être conservée à l'aide de l'argument `drop` :

```
# Sélection de la première colonne
x[, 1] # Retourne un vecteur
#> x y z
#> 1 2 3

x[, 1, drop = FALSE] # Retourne une matrice
#>   A
#> x 1
#> y 2
#> z 3
```

Avertissement

Un vecteur sans attribut `dim` n'a pas de dimension et ne doit pas être confondu avec une matrice à une seule ligne ou une seule colonne !

Les matrices sont des vecteurs possédant un attribut particulier (`dim`), il est donc également possible de sélectionner un sous-ensemble à l'aide d'un unique vecteur (souvenez-vous que les matrices sont remplies par colonnes) :

```
# Sélection des première et troisième valeurs
x[c(1, 3)]
#> [1] 1 3

# Remplacement des première et troisième valeurs
x[c(1, 3)] <- 0
x
#>   A B C  D
#> x 0 4 7 10
#> y 2 5 8 11
#> z 0 6 9 12
```

Enfin, il est possible d'utiliser une matrice pour sélectionner un sous-ensemble d'un tableau à deux dimensions ou plus. Cette matrice doit contenir les coordonnées (lignes et colonnes) des valeurs à sélectionner. La première colonne doit ainsi contenir les indices

des lignes à sélectionner, la seconde colonne les indices des colonnes et ainsi de suite pour les cas de dimensions supérieures (`array`) :

```
# On veut sélectionner les cellules (1,2), (2,3) et (3,4)
y <- matrix(c(1, 2, 3, 2, 3, 4), ncol = 2)

# La première colonne contient les indices des lignes
# La seconde colonne contient les indices des colonnes
y
#>      [,1] [,2]
#> [1,]    1    2
#> [2,]    2    3
#> [3,]    3    4

x[y]
#> [1]  4  8 12
```

3.4. Tableaux de données

Pour extraire ou remplacer des valeurs dans un `data.frame`, on peut utiliser deux stratégies. Les `data.frame` ont en effet les caractéristiques des listes et des matrices :

```
x <- data.frame(
  a = c(TRUE, FALSE, TRUE, TRUE),
  b = c(1, 2, 3, 4),
  c = c("A", "B", "C", "D")
)

# Comme une liste !
x[c("a", "b", "c")] # Colonnes nommées "a", "b" et "c"
#>       a b c
#> 1  TRUE 1 A
#> 2 FALSE 2 B
#> 3  TRUE 3 C
#> 4  TRUE 4 D
x[[2]]              # Deuxième colonne uniquement
#> [1] 1 2 3 4
x$b                 # Colonne nommée "b"
#> [1] 1 2 3 4

# Comme une matrice !
x[, c("a", "b", "c")]
```

```
#>        a b c
#> 1  TRUE 1 A
#> 2 FALSE 2 B
#> 3  TRUE 3 C
#> 4  TRUE 4 D
x[, 2]
#> [1] 1 2 3 4
x[, "b", drop = FALSE]
#>   b
#> 1 1
#> 2 2
#> 3 3
#> 4 4
```

4. Opérateurs binaires

4.1. Opérateurs arithmétiques

Les opérateurs arithmétiques (tab. 4.1) permettent de réaliser les opérations élémentaires :

```
x <- 1
y <- 2

# Addition
x + y
#> [1] 3

# Soustraction
x - y
#> [1] -1

# Multiplication
x * y
#> [1] 2

# Exponentiation
x^y
#> [1] 1
```

Tab. 4.1. : Les principaux opérateurs arithmétiques de R.

Opérateur	Description
`+`	addition
`-`	soustraction
`*`	multiplication
`/`	division
`^`	exponentiation

Une des particularités des R est de *vectoriser* de nombreuses opérations. Cela signifie que les opérations mathématiques sont réalisées élément par élément :

```
x <- 1:5
y <- 6:10

# Addition des éléments de x et y, un à un
# Équivaut à réaliser 1 + 6, 2 + 7, 3 + 8, 4 + 9 et 5 + 10
x + y
#> [1]  7  9 11 13 15
```

Cet aspect est important dans le cas d'opérations sur des matrices (souvenez-vous, une matrice est simplement un vecteur avec un attribut de dimension et est remplie colonne par colonne) :

```
# Une matrice de 5 lignes et 2 colonnes
x <- matrix(1:10, ncol = 2)
x
#>      [,1] [,2]
#> [1,]    1    6
#> [2,]    2    7
#> [3,]    3    8
#> [4,]    4    9
#> [5,]    5   10

# Un vecteur de 10 éléments
y <- 1:10
y
#>  [1]  1  2  3  4  5  6  7  8  9 10

# Les éléments de x et y sont additionnés un à un
x + y
#>      [,1] [,2]
#> [1,]    2   12
#> [2,]    4   14
#> [3,]    6   16
#> [4,]    8   18
#> [5,]   10   20
```

Si deux vecteurs n'ont pas la même longueur, R va **recycler** le plus court des deux et le réutiliser autant que nécessaire. Si la taille de l'objet le plus long n'est pas multiple de la taille de l'objet le plus court, un avertissement sera également retourné :

```
# Un vecteur de 5 éléments
i <- 1:5
i
```

```
#> [1] 1 2 3 4 5

# Les éléments de x et i sont additionnés un à un
# i est recyclé silencieusement
x + i
#>      [,1] [,2]
#> [1,]    2    7
#> [2,]    4    9
#> [3,]    6   11
#> [4,]    8   13
#> [5,]   10   15

# Un vecteur de 3 éléments
j <- 1:3
j
#> [1] 1 2 3

# Les éléments de x et j sont additionnés un à un
# j est recyclé avec un message d'avertissement
x + j
#> Warning in x + j: la taille d'un objet plus long n'est pas multiple
#> de la taille d'un objet plus court
#>      [,1] [,2]
#> [1,]    2    9
#> [2,]    4    8
#> [3,]    6   10
#> [4,]    5   12
#> [5,]    7   11
```

4.2. Opérateurs logiques

Les opérateurs logiques (tab. 4.2) permettent comparer des valeurs. Ces comparaisons se présentent sous la forme d'un test (la condition est-elle vraie ou fausse ?) et retournent donc un vecteur de type `logical` :

```
# x contient un unique élément
x <- 1

# La valeur de x est-elle égale à 0 ?
x == 0
#> [1] FALSE
```

```
# La valeur de x est-elle différente de 1 ?
x != 1
#> [1] FALSE

# La valeur de x est-elle supérieure à 0 ?
x > 0
#> [1] TRUE

# La valeur de x est-elle inférieure à 2 ?
x < 2
#> [1] TRUE
```

Tab. 4.2. : Les opérateurs logiques de R.

Opérateur	Comparaison
`>`	strictement supérieur
`>=`	supérieur ou égal
`<`	strictement inférieur
`<=`	inférieur ou égal
`==`	égal
`!=`	différent
`&`	ET
`|`	OU

Les opérateurs `&` (ET) et `|` (OU) permettent de combiner plusieurs conditions et de réaliser des comparaisons plus complexes. Dans ce cas, chaque condition est évaluée séparément, puis les résultats sont comparés :

```
# La valeur de x est-elle comprise entre 0 et 2 ?
x > 0 & x < 2
#> [1] TRUE

# La valeur de x est-elle égale à 0 ou 1 ?
x == 0 | x == 1
#> [1] TRUE

# ET
TRUE & TRUE
#> [1] TRUE
TRUE & FALSE
#> [1] FALSE
FALSE & FALSE
```

```
#> [1] FALSE

# OU
TRUE | TRUE
#> [1] TRUE
TRUE | FALSE
#> [1] TRUE
FALSE | FALSE
#> [1] FALSE

# NEGATION
!TRUE
#> [1] FALSE
!FALSE
#> [1] TRUE
```

Souvenez-vous, R vectorise de nombreuses opérations. La comparaison de deux objets est réalisée élément par élément :

```
x <- c(1, 4, 3, 6, 5)
y <- c(3, 2, 5, 4, 7)

# Comparaison des éléments de x et y, un à un
# Équivaut à tester si 1 < 3, 2 < 4, 3 < 5, 4 < 6 et 5 < 7
x < y
#> [1]  TRUE FALSE  TRUE FALSE  TRUE

# Toutes les valeurs de x sont-elles inférieures à y ?
all(x < y)
#> [1] FALSE

# Au moins une des valeurs de x est-elle inférieure à y ?
any(x < y)
#> [1] TRUE
```

Avertissement

Prenez garde aux valeurs manquantes ! Si une comparaison est réalisée avec une valeur manquante, le résultat sera une valeur manquante uniquement s'il est ambigu :

```
TRUE & NA
#> [1] NA
FALSE & NA
#> [1] FALSE

TRUE | NA
#> [1] TRUE
FALSE | NA
#> [1] NA
```

Utilisés conjointement avec `[` et `<-`, les opérateurs logiques permettent de filtrer et nettoyer facilement ses données :

```
x <- 0:10
x
#>  [1]  0  1  2  3  4  5  6  7  8  9 10

# Extraire les valeurs supérieures à 5
x[x > 5]
#> [1]  6  7  8  9 10

# Remplacer les 0 par des valeurs manquantes
x[x == 0] <- NA
x
#>  [1] NA  1  2  3  4  5  6  7  8  9 10
```

5. Structures de contrôle

5.1. Alternatives

Les alternatives permettent d'exécuter différentes instructions en fonction du résultat d'un test logique.

5.1.1. Test "si..."

L'instruction `if` permet d'exécuter un bloc de code, uniquement *si* une condition est vraie :

```
x <- 1

# On définit un test logique
# On encadre les instructions à exécuter avec des accolades
if (x > 0) {
  print("Texte affiché si la condition est vraie.")
}
#> [1] "Texte affiché si la condition est vraie."
```

5.1.2. Test "si... sinon..."

Il est possible de définir plusieurs choix en utilisant alternativement les instructions `if` et `else` :

```
x <- 1

# On définit un test logique
if (x < 0) {
  print("Texte affiché si la condition est vraie.")
} else {
  print("Texte affiché si la condition est fausse.")
}
#> [1] "Texte affiché si la condition est fausse."
```

Plusieurs alternatives peuvent être définies en utilisant conjointement `else` et `if`. On peut alors enchaîner autant de clauses "sinon si" que nécessaire et, éventuellement, définir une clause finale qui sera exécutée uniquement si aucune autre clause n'a été vérifiée :

```
x <- "chien"

if (x == "chat") {
  # Si la première condition est vraie
  message("x est un chat.")
} else if (x == "chien") {
  # Si la première condition est fausse et la seconde est vraie
  message("x est un chien.")
} else {
  # Si toutes les conditions sont fausses
  message("x est un animal inconnu.")
}
#> x est un chien.
```

5.1.3. Test "selon..."

L'instruction `switch` permet de choisir un bloc de code *selon* la valeur d'une variable.

L'exemple précédent peut être réécrit comme suit :

```
x <- "chien"

switch (
  x,
  chat = message("x est un chat."),
  chien = message("x est un chien."),
  message("x est un animal inconnu.")
)
#> x est un chien.
```

5.2. Boucles

Les boucles sont un moyen simple pour répéter des instructions en faisant varier un paramètre.

5.2.1. Boucles itératives

Une boucle itérative est contrôlée un compteur : une variable qui va prendre une valeur différente à chaque itération de la boucle. Le nombre d'exécutions de la boucle est connu à l'avance.

L'instruction `for` permet d'écrire des boucles en définissant un compteur et en spécifiant les instructions à répéter :

```
# On définit une variable i (le nom est libre)
# On précise que i va prendre successivement les valeurs 1 à 5
# On encadre les instructions à répéter avec des accolades
for (i in 1:5) {
  print(i) # A chaque itération, on affiche la valeur de i
}
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
#> [1] 5
```

Dans l'exemple suivant, on cherche à extraire et à afficher successivement chaque élément d'un vecteur :

```
# Création d'un vecteur contenant 5 éléments
x <- c(1.2, 6.5, 6.1, 5.9, 6.4)

# Faire varier i de 1 à 5
for (i in 1:5) {
  # Extraire la i-ème valeur de x
  # Stocker cette valeur dans l'objet k
  k <- x[i]
  # Afficher la valeur de k
  print(k)
}
#> [1] 1.2
#> [1] 6.5
#> [1] 6.1
#> [1] 5.9
#> [1] 6.4
```

Dans l'exemple précédent, la valeur de `k` a été remplacée à chaque itération de la boucle. Si on veut récupérer les résultats des différentes étapes, il va falloir au préalable créer un

objet vide susceptible de recevoir progressivement les différentes valeurs. La définition de cet objet implique d'anticiper les résultats de la boucle pour qu'il ait la structure adéquate :

```
# Créer un vecteur vide pour récupérer les résultats
# Le vecteur à la même longueur que le nombre d'itérations
k <- vector(mode = "numeric", length = 5)
k
#> [1] 0 0 0 0 0

# Faire varier i de 1 à 5
for (i in 1:5) {
  # Extraire la i-ème valeur de x
  # Affecter cette valeur à la i-ème position de k
  k[i] <- x[i]
}
k
#> [1] 1.2 6.5 6.1 5.9 6.4
```

Dans le cas d'un tableau à deux dimensions, écrire une même série d'instructions pour chaque ligne ou colonne individuellement peut rapidement s'avérer fastidieux. On peut ainsi mettre à profit les boucles pour n'écrire qu'une seule fois ces instructions et les appliquer à toutes les lignes ou toutes les colonnes.

Imaginons que l'on souhaite connaitre le nombre total de tessons de différents types de céramiques retrouvés sur un ensemble de sites archéologiques. Si nos données sont correctement structurées, on peut calculer cette somme pour chacune des colonnes d'un tableau de données[1] :

```
# On charge les données d'exemple
# help(compiegne, package = "folio")
data(compiegne, package = "folio")

# Le tableau possède 5 lignes et 16 colonnes
dim(compiegne)
#> [1]  5 16

# On crée un vecteur vide pour récupérer les résultats
# (16 valeurs sont attendues : la somme de chaque colonne)
resultats <- vector(mode = "numeric", length = 16)
```

[1] Il s'agit d'un exemple : dans le cas présent une boucle n'est pas nécessaire, R possède deux fonctions qui permettent de calculer la somme des lignes (`rowSums()`) ou des colonnes (`colSums()`) d'un tableau.

```
# On fait varier i de 1 à 16
for (i in 1:16) {
  # On extrait la i-ème colonne
  colonne_i <- compiegne[, i]
  # On calcule la somme de la colonne
  total_i <- sum(colonne_i)
  # On stocke la somme dans la i-ème position du vecteur de résultats
  resultats[i] <- total_i
}
resultats
#>  [1] 53200 12380 35300  8115  4825  7360  2775 10700   990  5425
#> [11] 18910 11155  2410  2835  1965  1185
```

5.2.2. Boucles de parcours

R possède une famille de fonctions qui permettent d'appliquer facilement des instructions aux différents éléments d'un objet. Les principales fonctions de cette famille sont :

- `apply()` : applique une fonction sur les lignes ou les colonnes d'une matrice (ou sur les dimensions supérieures d'un `array`).
- `lapply()` et `vapply()` : appliquent une fonction sur les différents éléments d'un vecteur ou d'une liste.
- `tapply()` : applique une fonction sur différents groupes de valeurs.

L'exemple précédent peut ainsi être simplifié avec `apply()` :

```
# Inutile de créer un objet vide au préalable
somme <- apply(
  X = compiegne,
  MARGIN = 2, # Appliquer aux lignes (1) ou aux colonnes (2)
  FUN = sum
)
somme
#>     A     B     C     D     E     F     G     H     I     J     K
#> 53200 12380 35300  8115  4825  7360  2775 10700   990  5425 18910
#>     L     M     N     O     P
#> 11155  2410  2835  1965  1185
```

Avertissement

La fonction `apply()` est conçue pour être utilisée sur une matrice (ou un `array`). Lorsqu'elle est utilisée sur un `data.frame`, elle va donc le convertir en `matrix`. Les éléments d'une matrice sont tous du même type : si les colonnes du `data.frame` sont de types différents, les règles de conversions implicites s'appliquent et peuvent entraîner des erreurs (par exemple lorsque les colonnes numériques sont transformées en chaînes de caractères, empêchant ainsi l'exécution d'une fonction mathématique).

Dans le cas d'un `data.frame`, il est préférable d'utiliser `lapply()` lorsqu'on souhaite appliquer une même fonction aux différentes colonnes :

```
x <- data.frame(
  a = c(3.6, 7.5, 7.2, 7.7, 4.8),
  b = c(5.8, 7.5, 1.5, 5.7, 2.3),
  c = c(7.2, 6.4, 3.3, 3.1, 8.1)
)

# Convertir toutes les colonnes en integer
y <- lapply(X = x, FUN = as.integer)
y
#> $a
#> [1] 3 7 7 7 4
#>
#> $b
#> [1] 5 7 1 5 2
#>
#> $c
#> [1] 7 6 3 3 8
```

`lapply()` retourne une simple liste, mais en utilisant une sélection vide, il est possible de conserver la structure du `data.frame` initial :

```
# Convertir toutes les colonnes en integer
# Conserve la structure du data.frame
# (mais remplace les données d'origine)
x[] <- lapply(X = x, FUN = as.integer)
x
#>   a b c
#> 1 3 5 7
#> 2 7 7 6
#> 3 7 1 3
```

```
#> 4 7 5 3
#> 5 4 2 8
```

`vapply()` est similaire à `lapply()`, mais peut être plus pratique à utiliser. `vapply()` simplifie le résultat final et retourne un vecteur ou, si cela est approprié, une matrice (comme pour `apply()`, faites attention aux règles de conversions implicites). De plus, le comportement de `vapply()` est prédictible : il est nécessaire de spécifier avec l'argument `FUN.VALUE` le type de résultat retourné.

```
# On spécifie le résultat de l'application de FUN aux éléments de X
# Celui-ci doit être un vecteur de type integer et de longueur 5
z <- vapply(X = x, FUN = as.integer, FUN.VALUE = integer(5))
z
#>      a b c
#> [1,] 3 5 7
#> [2,] 7 7 6
#> [3,] 7 1 3
#> [4,] 7 5 3
#> [5,] 4 2 8
```

Si le résultat de l'application de la fonction `FUN` à chaque élément de `X` ne correspond pas à la spécification, l'exécution du code est interrompue et une erreur est retournée :

```
# La longueur du vecteur retourné par FUN ne correspond pas
vapply(X = x, FUN = as.integer, FUN.VALUE = integer(3))
#> Error in vapply(X = x, FUN = as.integer, FUN.VALUE = integer(3)): les valeurs doivent être
#>  mais FUN(X[[1]]) a une longueur 5

# Le type du vecteur retourné par FUN ne correspond pas
vapply(X = x, FUN = as.integer, FUN.VALUE = character(5))
#> Error in vapply(X = x, FUN = as.integer, FUN.VALUE = character(5)): les valeurs doivent êt
#>  mais FUN(X[[1]]) est de type 'integer'
```

`tapply()` permet d'appliquer une même fonction à différents sous-ensembles d'un objet. Cette fonction est particulièrement pratique pour calculer des indicateurs statistiques par groupes :

```
# On dispose de 5 mesures d'un paramètre quelconque
valeurs <- c(6.8, 6.4, 5.8, 8.5, 8.8)
# Chaque mesure appartient à un groupe particulier (A ou B)
groupes <- c("A", "A", "B", "A", "B")
```

```
# On calcule la moyenne de chaque groupe
moy <- tapply(X = valeurs, INDEX = groupes, FUN = mean)
moy
#>        A        B
#> 7.233333 7.300000
```

5.2.3. Boucles à précondition

Une dernière possibilité, d'un usage peut-être moins courant, consiste à exécuter des instructions tant qu'une condition est vérifié. Cette boucle peut être considérée comme la répétition d'une instruction `if` *tant que* la valeur d'un test est vraie. Ainsi, une condition est vérifiée avant chaque itération de la boucle : si la condition est vraie (`TRUE`) les instructions sont exécutées, si la condition est fausse (`FALSE`) la boucle s'arrête sans exécuter les instructions.

```
# On définit la valeur initiale d'une variable
i <- 0

# On répète les instructions tant que i est inférieur à 5
while (i < 5) {
  print(i)   # Afficher la valeur de i
  i <- i + 1 # Incrémenter la valeur i
}
#> [1] 0
#> [1] 1
#> [1] 2
#> [1] 3
#> [1] 4
```

Remarque

Il convient d'être prudent avec l'usage de l'instruction `while`, car une mauvaise définition de la condition d'arrêt peut créer une boucle infinie.

Deuxième partie

Préparation des données

6. Importer

7. Nettoyer

8. Selectionner

9. Filtrer

10. Transformer

11. Exporter

Troisième partie

Éléments de statistiques

12. Statistique descriptive

13. Relations entre variables

14. Visualisation de données

Quatrième partie

Chronologie

15. Remarques préliminaires

En archéologie, le processus de datation consiste au "placement dans le temps d'événements les uns par rapport aux autres ou par rapport à toute échelle établie de mesure temporelle" (Dean, 1978). Cela implique de distinguer les méthodes de datation dites *relatives* (qui fournissent une séquence chronologique ordonnée d'événements) et les méthodes de datation dites *absolues* (qui donnent une indication calendaire et peuvent fournir la durée d'un événement) (O'Brien & Lyman, 2002).

À proprement parler, il n'existe pas de datation absolue, étant donné la façon dont les dates sont produites et que toute date s'inscrit dans un référentiel particulier. La distinction entre temps absolu et temps relatif peut être reformulée plus clairement comme quantifiable ou non quantifiable (O'Brien & Lyman, 2002) : les dates absolues "sont exprimées en points sur des échelles standard de mesure du temps" (Dean, 1978).

Nous garderons ici la distinction entre une *date* et un *âge* telle que formulée par Colman et al. (1987) : "une date est un point précis dans le temps, alors qu'un âge est un intervalle de temps mesuré en arrière à partir du présent". Traiter des dates en archéologie peut être délicat si l'on ne tient pas compte des sources de l'information chronologique. Dans la plupart des cas, une date représente un *terminus* pour un assemblage archéologique donné. C'est-à-dire une date avant (*terminus ante quem*) ou après (*terminus post quem*) laquelle l'événement étudié a eu lieu. Enfin, on gardera à l'esprit qu'il n'existe pas de datation sans incertitude en archéologie (même si cette dernière ne peut pas toujours être quantifiée).

16. Chronologie qualitative

17. Chronologie quantitative

Cinquième partie

Cartographie

Sixième partie

Annexes

A. Fonctions utiles

A.1. Séquences

```
# Séquence de nombres entiers
1:10
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```
# Séquence de 1 à 10
seq(1, 10)
#>  [1]  1  2  3  4  5  6  7  8  9 10

# Séquence de 1 à 10 par pas de 2
seq(1, 10, by = 2)
#> [1] 1 3 5 7 9
```

```
# Séquence de 10 entiers
seq_len(10)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

```
x <- c(8.6, 4.1, 3.9, 7, 1.8, 6.9, 1.6, 3.8, 2.6, 5.5)
length(x)
#> [1] 10

# Séquence de même longueur qu'un objet
seq_along(x)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

B. Bonnes pratiques

B.1. Travailler par projets

Une bonne pratique liée à l'utilisation de R consiste à organiser l'ensemble de son analyse en un projet, c'est à dire un dossier autonome contenant l'ensemble des fichiers nécessaires à l'analyse (dont les données brutes et le fichiers générés). Une telle organisation doit permettre de déplacer le dossier sur son ordinateur ou sur un autre ordinateur, tout en maintenant le bon fonctionnement du code (absence de liens brisés, de dépendances manquantes…).

Ce dossier, ou répertoire de travail (*working directory*), peut être défini ou retrouvé à l'aide des fonctions `setwd()` et `getwd()`, respectivement :

```
# Définition du répertoire de travail
setwd("/chemin/vers/mon/dossier")
```

Une fois le répertoire de travail défini, toute référence à un fichier en utilisant un simple nom de fichier ou un chemin relatif sera interprétée relativement au répertoire de travail.

Cependant, le risque est de se retrouver dans la situation suivante :

```
# Définition du répertoire de travail
setwd("/chemin/uniquement/valide/sur/mon/ordinateur")
```

Pour pallier ce problème qui rend difficilement déplaçable un dossier de projet, il existe différentes solutions parmi lesquelles :

- L'utilisation du package *here*,
- L'utilisation de la gestion de projet dans RStudio.

RStudio dispose d'un mécanisme permettant de créer facilement de tels projets. Il est possible de créer un projet dans RStudio à partir du menu déroulant situé en haut à droite de l'interface ou en cliquant sur *New Project…* depuis le menu *File*. Le projet peut alors être créé soit dans un nouveau dossier, soit en transformant un dossier existant (fig. B.1).

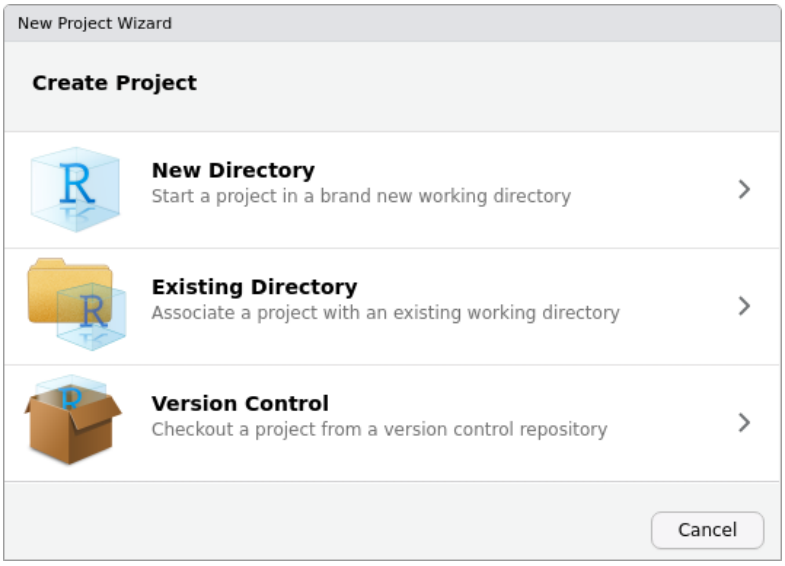

FIG. B.1. : Création d'un projet avec RStudio.

Un projet créé par RStudio est reconnaissable par la présence d'un fichier `.Rproj`. Ce fichier marque le dossier de plus haut niveau au sein d'un projet (répertoire de travail), à partir duquel des chemins d'accès relatifs peuvent être utilisés pour lire ou écrire des fichiers. Le répertoire de travail est automatiquement définit lors de l'ouverture d'un projet RStudio : il n'est plus nécessaire d'utiliser `setwd()`.

L'organisation des fichiers et des sous-dossiers au sein d'un projet relève des habitudes de travail de chacun. S'il n'existe pas de consensus sur la manière d'organiser les fichiers à l'intérieur d'un projet, il peut cependant être avantageux de suivre certaines conventions (fig. B.2), comme celles utilisées par le package *rrtools* par exemple.

B.2. Structurer ses données

Lors de l'acquisition puis lors de la préparation de vos données en vue d'une étude, toutes vos actions doivent être guidées par un impératif : structurer au mieux vos jeux de données pour en faciliter l'analyse (WICKHAM, 2014). Les principes régissant cette étape de structuration (*data tidying*) ont été exposés par WICKHAM (2014), qui propose ainsi une approche pour lier "la structure d'un jeu de données (sa mise en forme) avec sa dimension sémantique (sa signification)."

Des jeux de données bien structurés se présentent sous la forme de tableaux à deux dimensions (lignes et colonnes) contenant des valeurs. Chaque valeur correspond à une observation d'une variable particulière, en conséquence :

- Chaque variable doit correspondre à une colonne du tableau.

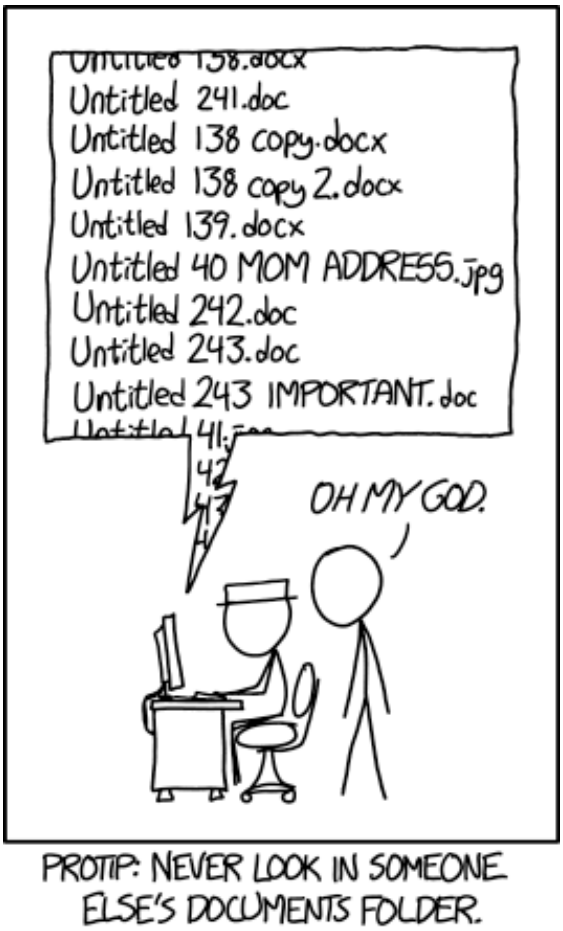

Fig. B.2. : Documents. "Copy of Copy of Copy of

- Chaque observation doit correspondre à une ligne du tableau.
- Un tableau doit correspondre à une unique unité d'observation[1].

B.3. Sauvegarder ses données

Une fois vos données correctement structurées, choisissez un format de fichier adapté pour les archiver, les diffuser ou les réutiliser (fig. B.3). De manière générale, conservez vos données dans un fichier texte.

Dans le cas des données tabulaires, utilisez le format CSV. Comme son nom l'indique, un fichier CSV est un fichier texte contenant des valeurs séparées par une virgule. Cette simplicité lui confère plusieurs avantages, ce qui en fait le format idéal pour pérenniser des données :

- Un fichier CSV est une suite de caractères et de retours à la ligne : il est facilement éditable et peut aussi bien être lu par un humain (les données ne sont pas encodées) que par une machine.
- Le format CSV est un format ouvert : son utilisation ne dépend pas d'un éditeur particulier. Un fichier CSV peut ainsi être lu ou écrit par n'importe quel logiciel capable de manipuler des feuilles de calcul (ou par un simple éditeur de texte).
- Le format CSV est un format bien établi (ses origines remontent aux années 1970), il ne risque pas de subir un changement dramatique de spécification.
- Son apparente austérité (il ne contient pas de formatage) oblige a structurer correctement ses données.

La simplicité et la souplesse de ce format nécessitent cependant un peu de prudence au moment de la création d'un fichier CSV. Il est en effet possible d'utiliser n'importe quel caractère en guise de séparateur à la place d'une virgule. Si votre tableur est paramétré en français, par défaut, les données seront séparées un point-virgule pour éviter les confusions avec le séparateur décimal (de même, lors de l'ouverture d'un fichier CSV votre éditeur s'attendra à trouver des valeurs séparées par un point-virgule). Pensez à bien paramétrer votre logiciel lors de l'import ou de l'export d'un fichier CSV dans un tableur (fig. B.4).

Attention

Méfiez-vous de votre tableur ! Ces derniers ont tendance à réaliser automatiquement des conversions qui ne sont pas sans conséquences lors de l'ouverture d'un fichier.

[1] On reconnait ici la troisième forme normale, dans le cas des bases de données relationnelles (Wickham, 2014).

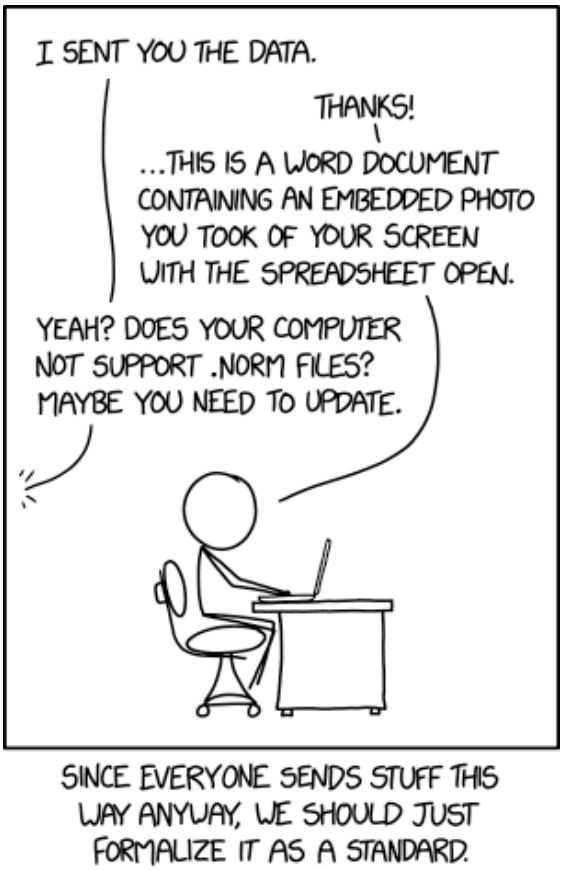

Fig. B.3. : .NORM Normal File Format. “At some point, compression becomes an aesthetic design choice. Luckily, SVG is a really flexible format, so there’s no reason it can’t support vector JPEG artifacts.” (CC BY-NC 2.0 Munroe, 2019).

FIG. B.4. : Paramètres lors de l'export d'un fichier au format CSV avec LibreOffice Calc (choisissez l'encodage de caractères UTF-8 et un séparateur de champ adapté).

B.4. Limiter les dépendances

Si les packages de base de R offrent de nombreuses possibilités, il est courant d'avoir besoin de fonctionnalités supplémentaires au cours d'une étude. Pour une analyse spécifique, il est très probable qu'il existe déjà un ou plusieurs packages offrant les fonctionnalités recherchées et installable depuis le CRAN. Cette offre pléthorique a cependant un revers : à chaque package supplémentaire utilisé dans votre projet, vous augmentez le risque de voir apparaître des problèmes liés à ces dépendances[2].

Par exemple, *FactoMineR* est sans doute le package le plus utilisé pour l'analyse de données multivariées. *FactoMineR* possède 15 dépendances directes : d'autres packages dont il utilise les fonctionnalités. Cependant, chacune de ces dépendances est susceptible d'avoir elle même des dépendances, et ainsi de suite, si bien que *FactoMineR* a en réalité une longue chaîne de 103 (!) dépendances (fig. B.5).

Qu'arrivera-t-il alors si une seule des ces dépendances change drastiquement, arrête de fonctionner ou disparaît tout simplement (fig. B.6) ? Pour réduire ce risque et sortir de cet enfer des dépendances :

- Évitez d'utiliser un package particulier quand la même tâche peut être réalisée en R basique (écrivez vos propres fonctions !).
- Quand cela est possible, préférez les packages qui n'ont pas (ou peu) de dépendances.
- N'utilisez pas la version de développement d'un package, mais installez toujours la version stable depuis le CRAN.

[2]Sur le sujet, voir les billets regroupés sur le site du tinyverse.

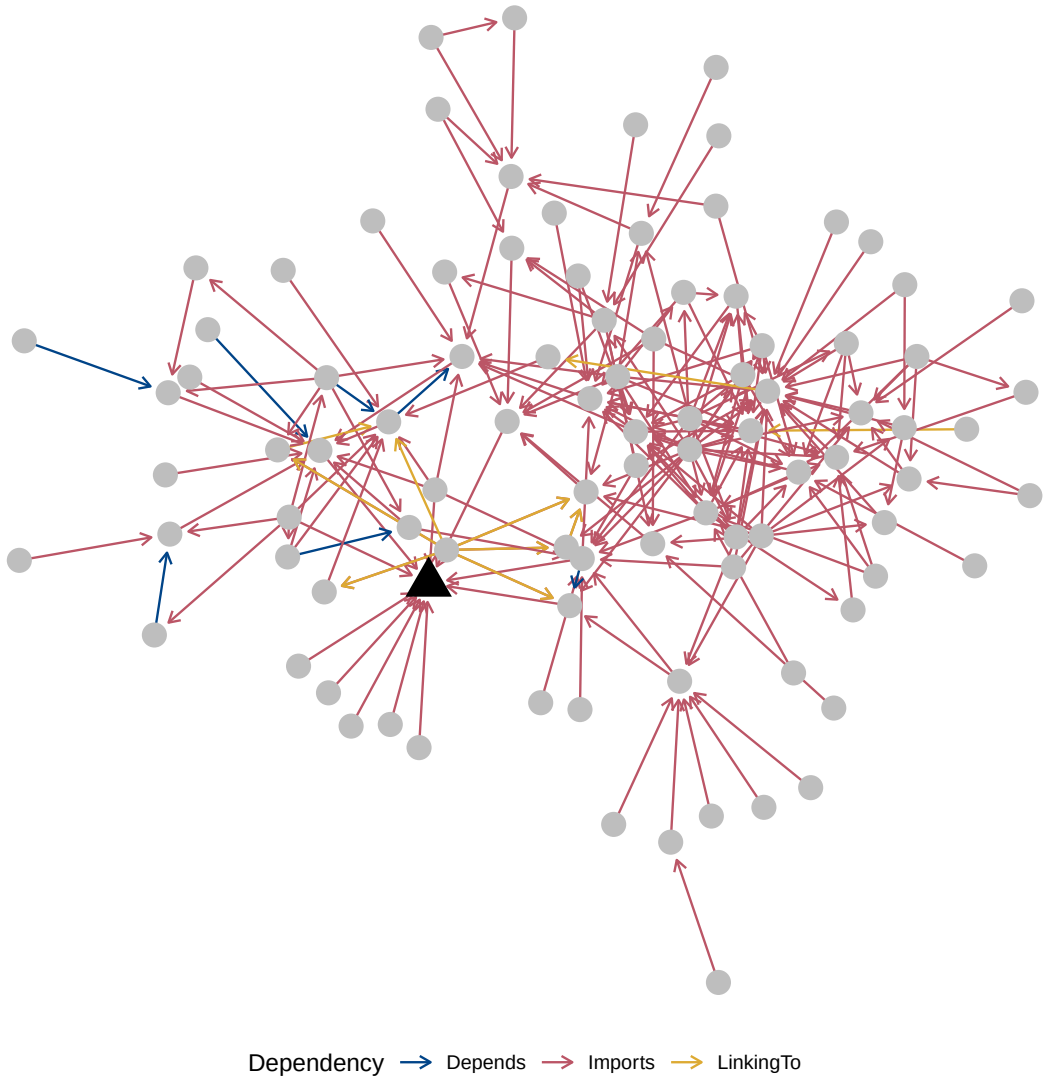

FIG. B.5. : Réseau des dépendances du package *FactoMineR* (hors packages de base). Les noms des packages ont été omis pour faciliter la lecture (*FactoMineR* est représenté par un triangle noir, les autres packages sont représentés par des points gris).

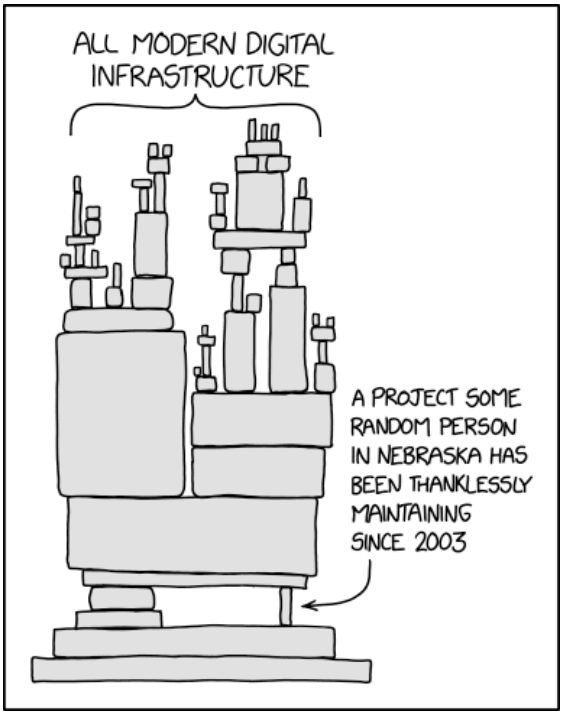

Fig. B.6. : Dependency. “Someday ImageMagick will finally break for good and we’ll have a long period of scrambling as we try to reassemble civilization from the rubble.” (CC BY-NC 2.0 Munroe, 2020).

Le tableau n'est cependant pas totalement noir. Les packages publiés sur le CRAN doivent se conformer à des règles strictes et sont continuellement testés dans différentes configurations (systèmes d'exploitation et versions de R), obligeant les développeurs à réagir rapidement lorsqu'un bug est observé. De plus, lorsqu'un package n'est plus disponible sur le CRAN, les versions antérieures sont archivées et restent disponibles au téléchargement. Enfin, des initiatives comme rOpenSci œuvrent pour garantir un écosystème fonctionnel, en favorisant l'évaluation et la maintenance des packages.

B.5. Écrire des exemples reproductibles

Si vous souhaitez obtenir de l'aide en ligne auprès de la communauté des utilisateurs de R, sur Stack Overflow[3] ou sur les listes de diffusion, la seule description de votre difficulté ne sera pas suffisante. Vous devez permettre aux autres de reproduire le problème sur leur machine pour qu'ils puissent vous proposer une solution. Pour cela, inutile de diffuser l'intégralité de votre code et de vos données, préparez un exemple qui soit :

- Minimal : utilisez le moins de code possible tout en produisant le même problème.
- Complet : fournissez tous les éléments (version de R, packages utilisés, etc.) dont un tiers a besoin pour reproduire votre problème. Utilisez les données d'exemple de R.
- Reproductible : le code que vous vous apprêtez à fournir doit reproduire le problème.

Ces trois aspects sont détaillés dans l'aide de Stack Overflow, il existe également une question dédié à l'écriture d'un exemple reproductible (*reprex*) avec R.

Conseil

L'utilisation du package *reprex*[a] peut vous aider à écrire un exemple reproductible.

[a] Voir https://reprex.tidyverse.org/articles/articles/learn-reprex.html.

Comme le souligne Wickham (2016), la plupart du temps, l'écriture d'un exemple reproductible vous permettra d'identifier et de résoudre vous-même le problème.

[3] Avant de poster votre question sur Stack Overflow, prenez le temps de bien lire les recommandations d'écriture.

C. Style de programmation

Cette section propose un style de programmation destiné à accompagner les utilisatrices et utilisateurs débutants. Ce style regroupe des recommandations pour l'écriture du code et permet d'en faciliter la lecture et la compréhension. Ce style est dérivé du *tidyverse style guide* et du livre *rOpenSci Packages*.

C.1. Dénominations

En règle générale, les abréviations doivent être évitées lors de la dénomination de fichiers ou d'objets.

C.1.1. Nommer les fichiers

Les noms de fichiers doivent utiliser l'extension `.R`.

```
# GOOD
plot.R

# BAD
plot
```

Les noms de fichiers doivent être parlants.

```
# GOOD
plot.R

# BAD
Untitled1.R
```

Les noms de fichiers ne doivent pas contenir de `/` et d'espaces. À la place, utilisez un tiret (`-`) ou un trait de soulignement (`_`).

```
# GOOD
read_csv.R
plot-methods.R

# BAD
read csv.R
```

Les noms de fichiers doivent utiliser des lettres du latin de base, et non du supplément latin-1, et doivent être en minuscules.

```
# GOOD
plot.R

# BAD
Plot.R
données.R
```

Utilisez des verbes signifiants pour les noms de fichiers.

```
# GOOD
fit_model.R

# BAD
addition.R
```

Si le fichier ne contient qu'une seule fonction, nommez-le par le nom de la fonction.

C.1.2. Nommer les variables

Les noms des variables doivent être aussi courts que possible.

Les noms des variables doivent être des substantifs évocateurs.

Les noms de variables doivent être en minuscules.

Ne séparez jamais les mots à l'intérieur du nom par `.` (réservé pour les méthodes S3) et n'écrivez pas en PascalCase (réservé pour les définitions de classes S4). Utilisez plutôt un trait de soulignement (`_`).

```
# GOOD
std_dev <- 3

# BAD
std.dev <- 3
StdDev <- 3
```

N'utilisez pas les noms de fonctions et de variables existantes (en particulier celles qui sont propres à R).

```
# GOOD
std_dev <- 3

# BAD
T <- 1
c <- 2 * 2
mean <- 10
```

C.1.3. Nommer les fonctions

Les noms de fonction doivent contenir un verbe qui fait référence à l'action principale de la fonction.

Les noms de fonction doivent être en snake_case. N'utilisez `.` que pour le dispatching des méthodes S3.

Le schéma de nommage `object_verb()` doit être préféré aussi souvent que possible. Ce schéma est facile à lire et à compléter automatiquement.

```
# GOOD
peak_detect()

# BAD
addition()
readFile()
```

Évitez de réemployer des noms de fonctions déjà utilisés par les packages de base ou d'autres packages populaires.

C.1.4. Nommer les classes S4

Les noms des classes doivent être des substantifs en PascalCase avec une majuscule initiale et la première lettre de chaque mot concaténé en majuscule.

C.2. Syntaxe

C.2.1. Longueur de ligne

La longueur maximale des lignes est limitée à 80 caractères.

Ne mettez pas plus d'une déclaration par ligne. N'utilisez pas de point-virgule pour terminer la commande.

```
# GOOD
x <- 1
x <- x + 1

# BAD
x <- 1; x <- x + 1
```

C.2.2. Appel de fonction

Dans un appel de fonction, spécifiez les arguments par leur nom. Ne spécifiez jamais les arguments par un nom partiel et ne mélangez jamais la position et le nom complet.

```
# GOOD
mean(x, na.rm = TRUE)

# BAD
mean(x, na = TRUE)
```

Les arguments obligatoires doivent être placés en premier, suivis des arguments facultatifs.

```
# GOOD
raise_to_power(x, power = 2.7)

# BAD
raise_to_power(power = 2.7, x)
```

L'argument `...` doit se trouver soit au début, soit à la fin.

```
# GOOD
standardize(..., scale = TRUE, center = TRUE)

# BAD
standardize(scale = TRUE, ..., center = TRUE)
```

Définissez les arguments par défaut à l'intérieur de la fonction en utilisant l'idiome `NULL`, et évitez la dépendance entre les arguments.

Validez toujours les arguments dans une fonction.

C.2.3. Espaces de nom

Si vous utilisez ponctuellement une unique fonction d'un package, utilisez `::` pour spécifier le nom du package au lieu de charger tout le package.

```
# GOOD
FactoMineR::PCA()

# OK
library(FactoMineR)
PCA()
```

L'utilisation des espaces de nom permet d'éviter les conflits lorsque deux packages ont une fonction du même nom et explicite l'origine de la fonction utilisée.

C.2.4. Boucles

Utilisez `seq_along()` dans les boucles pour gérer correctement les cas où `x` est vide.

```
# Soit un vecteur numérique de longueur nulle
x <- numeric(0)

# La boucle ne s'exécute pas (OK)
for (i in seq_along(x)) {
  print(i)
}

# La boucle s'exécute pour i=1 puis i=0 (!)
for (i in 1:length(x)) {
  print(i)
}
#> [1] 1
#> [1] 0
```

C.2.5. Affectation

Utilisez `<-` au lieu de `=` pour l'affectation.

```
# GOOD
x <- 1

# BAD
x = 1
1 -> x
```

C.2.6. Espaces

Mettez des espaces autour de tous les opérateurs binaires infixes (`=`, `+`, `*`, `==`, `&&`, `<-`, `%*%`, etc.).

```
# GOOD
x == y
z <- 2 + 1

# BAD
x==y
z<-2+1
```

Mettez des espaces autour de `=` dans les appels de fonction.

```
# GOOD
mean(x = c(1, 2, 3), na.rm = TRUE)

# BAD
mean(x=c(1, 2, NA), na.rm=TRUE)
```

Ne placez pas d'espace pour obtenir des sous-ensembles (`$` et `@`), pour la manipulation des espaces de noms (`::` et `:::`) et pour la génération de séquences (`:`).

```
# GOOD
car$cyl
dplyr::select()
1:10

# BAD
car $cyl
dplyr:: select()
1: 10
```

Mettez un espace après une virgule.

```
# GOOD
mtcars[1, ]
mean(x = c(1, NA, 2), na.rm = TRUE)

# BAD
mtcars[1 ,]
mean(x = c(1,NA,2),na.rm = TRUE)
```

Utilisez un espace avant les parenthèses ouvrantes, sauf dans un appel de fonction.

```
# GOOD
for (element in element_list)
if (total == 5)
sum(1:10)

# BAD
for(element in element_list)
if(total == 5)
sum (1:10)
```

Ne mettez pas d'espaces autour du code entre parenthèses ou entre crochets.

```
# GOOD
if (is_true) message("Hello!")
species["tiger", ]

# BAD
if ( is_true ) message("Hello!")
species[ "tiger" ,]
```

C.2.7. Accolade

Une accolade ouvrante ne doit jamais être placée sur sa propre ligne et doit toujours être suivie d'une nouvelle ligne.

```
# GOOD
if (is_true) {
  # do something
}
```

```
if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true)
{
  # do something
}

if (is_true) { # do something }
else { # do something else }
```

Une accolade fermante doit toujours être placée sur sa propre ligne, sauf si elle est suivie de `else`.

```
# GOOD
if (is_true) {
  # do something
} else {
  # do something else
}

# BAD
if (is_true) {
  # do something
}
else {
  # do something else
}
```

Indentez toujours le code à l'intérieur des accolades.

```
# GOOD
if (is_true) {
  # do something
  # and then something else
}

# BAD
```

```
if (is_true) {
  # do something
  # and then something else
}
```

Les accolades et les nouvelles lignes peuvent être évitées, si une déclaration après `if` est très courte.

```
# OK
if (is_true) return(value)
```

C.2.8. Indentation

N'utilisez pas de tabulations ou de mélanges de tabulations et d'espaces pour l'indentation.

Utilisez deux espaces pour l'indentation.

C.2.9. Nouvelle ligne

Dans une définition ou un appel de fonction, les arguments excessifs doivent être indentés sur la ligne de la parenthèse fermante, si deux lignes suffisent.

```
# GOOD
long_function_name <- function(arg1, arg2, arg3, arg4,
                               long_argument_name1 = TRUE)

plot(table(rpois(100, 5)), type = "h", col = "red", lwd = 10,
     main = "rpois(100, lambda = 5)")
```

Sinon, chaque argument peut aller sur une ligne séparée, en commençant par une nouvelle ligne après la parenthèse ouvrante.

```
# GOOD
long_function_name <- function(long_argument_name1 = c("value1", "value2"),
                               long_argument_name2 = TRUE,
                               long_argument_name3 = NULL,
                               long_argument_name4 = FALSE)

list(
  mean = mean(x),
```

```
  sd = sd(x),
  var = var(x),
  min = min(x),
  max = max(x),
  median = median(x)
)
```

Si la condition dans l'instruction `if` s'étend sur plusieurs lignes, alors chaque condition doit se terminer par un opérateur logique, et non commencer par lui.

```
# GOOD
if (some_very_long_name_1 == 1 &&
    some_very_long_name_2 == 1 ||
    some_very_long_name_3 %in% some_very_long_name_4)

# BAD
if (some_very_long_name_1 == 1
    && some_very_long_name_2 == 1
    || some_very_long_name_3 %in% some_very_long_name_4)
```

Si l'instruction qui contient des opérateurs s'étend sur plusieurs lignes, chaque ligne doit se terminer par un opérateur et non commencer par lui.

```
# GOOD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2) *
  exp(-(x - d_mean) ^ 2 / 2 / s ^ 2)

# BAD
normal_pdf <- 1 / sqrt(2 * pi * d_sigma ^ 2)
* exp(-(x - d_mean) ^ 2 / 2 / d_sigma ^ 2)
```

Chaque appel de `dplyr` (après `%>%` ou `|>`) ou de `ggplot2` (après `+`) doit commencer sur une nouvelle ligne.

C.3. Commentaires

Il est recommandé d'écrire les commentaires en anglais.

Les commentaires commencent par `#` suivi d'un espace et du texte du commentaire.

```
# This is a comment.
```

Les commentaires doivent expliquer le pourquoi, et non le comment. Les commentaires doivent expliquer l'intention générale de la commande.

```
# GOOD
# define iterator
i <- 1

# BAD
# set i to 1
i <- 1
```

De courts commentaires peuvent être placés sur la même ligne que le code code.

```
plot(price, weight) # Plot a scatter chart of price and weight
```

Il peut être utile de diviser le code source en morceaux logiques par un commentaire suivi de `-` ou `=`.

```
# Read data ------------------------------------------------------------

# Clean data -----------------------------------------------------------
```

Les descriptions des fonctions et des objets doivent respecter les directives de [*roxygen2*] (https://roxygen2.r-lib.org/).

```
#' Add Together Two Numbers
#'
#' @param x A number.
#' @param y A number.
#' @return The sum of x and y.
#' @examples
#' add(1, 1)
#' add(10, 1)
add <- function(x, y) {
  # general comment
  x + y # inline comment
}
```

Bibliographie

Adler, J. (2010). *R in a nutshell*. O'Reilly.

Allaire, J., Horner, J., Xie, Y., Marti, V., & Porte, N. (2019). *markdown : Render Markdown with the C Library Sundown* [R package version 1.1]. https://github.com/rstudio/markdown

Allaire, J., Xie, Y., McPherson, J., Luraschi, J., Ushey, K., Atkins, A., Wickham, H., Cheng, J., Chang, W., & Iannone, R. (2021). *rmarkdown : Dynamic Documents for R* [R package version 2.11]. https://CRAN.R-project.org/package=rmarkdown

Barnier, J. (2021). *Introduction à R et au tidyverse*. https://juba.github.io/tidyverse

Bryan, J. (2020). *STAT 545 : Data wrangling, exploration, and analysis with R*. https://stat545.com/

Bryan, J. (2021). *Happy Git and GitHub for the useR*. https://happygitwithr.com/

Chang, W. (2012). *R graphics cookbook*. O'Reilly Media.

Chang, W. (2019). *webshot : Take Screenshots of Web Pages* [R package version 0.5.2]. https://github.com/wch/webshot/

Colman, S. M., Pierce, K. L., & Birkeland, P. W. (1987). Suggested Terminology for Quaternary Dating Methods. *Quaternary Research*, *28*(2), 314-319. https://doi.org/10.1016/0033-5894(87)90070-6

Csárdi, G., Müller, K., & Hester, J. (2021). *desc : Manipulate DESCRIPTION Files* [R package version 1.4.0]. https://github.com/r-lib/desc#readme

Dean, J. S. (1978). Independent Dating in Archaeological Analysis. In *Advances in Archaeological Method and Theory* (p. 223-255). Elsevier. https://doi.org/10.1016/B978-0-12-003101-6.50013-5

Frerebeau, N. (2021). *folio : Datasets for Teaching Archaeology and Paleontology* [R package version 1.0.0]. https://CRAN.R-project.org/package=folio

Gillespie, C. (2016). *Efficient R programming : A practical guide to smarter programming*. O'Reilly Media.

Grolemund, G. (2015). *Hands-on Programming with R*. O'Reilly.

Iannone, R. (2022). *DiagrammeR : Graph/Network Visualization* [R package version 1.0.8]. https://github.com/rich-iannone/DiagrammeR

Long, J. D. (2019). *R cookbook : Proven recipes for data analysis, statistics, and graphics*. O'Reilly Media.

Lovelace, R. (2020). *Geocomputation with r*. CRC Press.

Matloff, N. (2019). *An opinionated view of the Tidyverse "dialect" of the R language*. TidyverseSkeptic. Récupérée 12 février 2022, à partir de https://github.com/matloff/TidyverseSkeptic

Millot, G. (2014). *Comprendre et réaliser les tests statistiques à l'aide de R - Manuel de biostatistique* (troisième édition). De Boeck.

Munroe, R. (2014, décembre 12). *Documents*. xkcd. https://xkcd.com/1459/

Munroe, R. (2019, février 25). *.NORM Normal File Format*. xkcd. https://xkcd.com/2116/

Munroe, R. (2020, août 17). *Dependency*. xkcd. https://xkcd.com/2347/

Murrell, P. (2019). *R graphics*. CRC Press, Taylor & Francis Group.

O'Brien, M. J., & Lyman, R. L. (2002). *Seriation, Stratigraphy, and Index Fossils : The Backbone of Archaeological Dating*. Springer.

R Core Team. (2021). *R : A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria. https://www.R-project.org/

Schmidt, S. C., & Marwick, B. (2020). Tool-Driven Revolutions in Archaeological Science. *Journal of Computer Applications in Archaeology*, *3*(1), 18-32. https://doi.org/10.5334/jcaa.29

Sievert, C., & Cheng, J. (2021). *bslib : Custom Bootstrap 'Sass' Themes for shiny and rmarkdown* [R package version 0.3.1]. https://CRAN.R-project.org/package=bslib

Silge, J. (2017). *Text mining with R : A tidy approach*. O'Reilly Media.

Thieme, N. (2018). R generation. *Significance*, *15*(4), 14-19. https://doi.org/10.1111/j.1740-9713.2018.01169.x

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, *59*(10), 1-23. https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2015). *R packages*. O'Reilly Media.

Wickham, H. (2016). *R for data science : Import, tidy, transform, visualize, and model data*. O'Reilly Media.

Wickham, H. (2019). *Advanced r*. Taylor & Francis, CRC Press.

Wickham, H. (2021). *downlit : Syntax Highlighting and Automatic Linking* [R package version 0.4.0]. https://CRAN.R-project.org/package=downlit

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wickham, H., Chang, W., Flight, R., Müller, K., & Hester, J. (2021). *sessioninfo : R Session Information* [R package version 1.2.2]. https://CRAN.R-project.org/package=sessioninfo

Wickham, H., Hester, J., & Ooms, J. (2021). *xml2 : Parse XML* [R package version 1.3.3]. https://CRAN.R-project.org/package=xml2

Wilke, C. O. (2019). *Fundamentals of data visualization : A primer on making informative and compelling figures*. O'Reilly Media, Inc.

Xie, Y. (2016). *bookdown : Authoring Books and Technical Documents with R Markdown* [ISBN 978-1138700109]. Chapman ; Hall/CRC. https://bookdown.org/yihui/bookdown

Xie, Y. (2021a). *bookdown : Authoring Books and Technical Documents with R Markdown* [R package version 0.24]. https://CRAN.R-project.org/package=bookdown

Xie, Y. (2021b). *knitr : A General-Purpose Package for Dynamic Report Generation in R* [R package version 1.37]. https://yihui.org/knitr/

Xie, Y. (2021c). *mime : Map Filenames to MIME Types* [R package version 0.12]. https://github.com/yihui/mime